INFORME DE SEGURIDAD

Puerta 7
Revisión de los contratos

Qué estaba mal, cómo se podía aprovechar,
cómo se corrigió y cómo explicarlo

15 fallas corregidas en dos rondas · evidencia de ejecución en un nodo local
Rama `gperez` · base `fb59636` ("front e integracion final")
Contratos: Entrada · Sorteo · Reventa · Canje · Puntos
Octubre 2026

Contenido

1. Cómo leer este informe
2. Resumen ejecutivo
3. Conceptos que necesitás antes de empezar
4. El sistema en un dibujo: los 5 contratos
5. F1 · El sorteo se podía trabar para siempre ALTA
6. F2 · El sorteo se podía reabrir ALTA
7. F3 · La comisión de reventa se podía cambiar a último momento MEDIA
8. F4 · "Renunciar" dejaba la plata atrapada MEDIA
9. F5 · El depósito de más quedaba atrapado BAJA
10. F6 · Inyección en la imagen y los datos del NFT BAJA
11. Segunda ronda: las correcciones que pidió el profesor
    1. N1 · Comprar y revertir hasta elegir el asiento MEDIA
    2. N2 · El último en revelar podía calcular el resultado MEDIA
    3. N3 · El portero podía marcar entradas el día antes MEDIA
    4. R1 · Plata atrapada en el sorteo MEDIA
    5. R3 · Traspaso de dueño a una dirección mal tipeada MEDIA
    6. R2 · Un sorteo demasiado grande para cerrarse BAJA
    7. R4 · Un operador podía quemar entradas ajenas BAJA
    8. R5 · El listado viejo que revivía BAJA
    9. N6 · La autorización de puntos que nunca se gastaba BAJA
12. Lo que sigue sin poder corregirse
13. Evidencia de ejecución (capturas)
14. Cómo se verificó todo (tests)
15. Decisiones que tomé (y se pueden revisar)
16. Cómo presentarlo: guía de 6 minutos y preguntas frecuentes
17. Glosario

1. Cómo leer este informe

Este informe cuenta una revisión de seguridad de los contratos de Puerta 7. Está escrito para que alguien sin experiencia previa en smart contracts lo entienda de punta a punta y después lo pueda explicar a otros. Si un término no te suena, está en el capítulo 3 o en el glosario del final.

Todos los hallazgos usan la misma estructura

Parte	Qué responde
En una frase	El problema resumido para alguien que no lee código.
Contexto	Qué hace esa parte del contrato cuando todo funciona bien.
Qué estaba mal	La causa técnica exacta, con el código.
Cómo se explotaba	Un ataque o un accidente concreto, paso a paso, con números.
La corrección	Qué se cambió, con el código de antes y de después.
Cómo lo verificamos	El test que demuestra el problema y la corrección.
Cómo explicarlo	Una forma corta de contarlo en una presentación.

Qué significa cada severidad

La severidad combina dos preguntas: **¿cuánto daño hace?** y **¿qué tan fácil es que pase?**

Severidad	Significado en este informe
ALTA	Puede hacer perder plata de los usuarios o romper una regla central del sistema, y pasa sin que haga falta nada raro (o un atacante lo puede provocar).
MEDIA	Daño grande, pero necesita que alguien con poder (el organizador) se equivoque o actúe de mala fe.
BAJA	Daño acotado, o solo lo puede provocar alguien con poder y en casos poco comunes.
DISEÑO	Una limitación conocida del diseño que no se puede (o no se debe) arreglar en el contrato.

Colores de los recuadros

> Azul: aclaraciones.

> Amarillo: advertencias o límites.

> Rojo: el impacto del problema.

Verde: la corrección o un dato a favor.

Violeta: cómo explicarlo en voz alta.

2. Resumen ejecutivo

Qué se hizo

1. **Pull** de la versión nueva del repositorio. Trajo 3 commits con cuatro contratos nuevos (Sorteo, Reventa, Canje, Puntos), el front completo y una versión reescrita de `Entrada.sol`.
2. **Revisión** de los 5 contratos, del script de despliegue y de los scripts, más una búsqueda de secretos (claves privadas, contraseñas) en el front. No se encontró ningún secreto subido.
3. **Primera ronda:** corrección de 6 fallas (F1–F6), cada una con un test que primero demostró el problema (rojo) y después la corrección (verde).
4. **Segunda ronda**, a pedido del profesor ("no basta con detectarlas, se deben corregir en el código"): corrección de 9 hallazgos más (N1–N3, N6, R1–R5) que la primera versión solo documentaba. Incluye cambios de diseño: la compra y el cierre del sorteo pasan a tener dos pasos.
5. **Evidencia de ejecución:** despliegue del sistema completo en un nodo local, recorrido de punta a punta contra el nodo y una demo de ataques que se corre contra el código subido y contra el corregido (capítulo 13).

2	7	6	2	160
ALTAS	MEDIAS	BAJAS	SIN ARREGLO ON-CHAIN	TESTS EN VERDE

La demo, en una línea: el mismo script de ataques logra **15 de 15** contra el código subido y **0 de 15** contra el corregido.

Primera ronda

#	Severidad	Contrato	Problema	Corrección
F1	ALTA	Sorteo	Con muchos participantes, cerrar el sorteo costaba más gas del que entra en un bloque: nadie podía cerrarlo y todos los depósitos quedaban congelados.	El costo de cerrar ahora depende solo de cuántas entradas se sortean, no de cuántos participan.
F2	ALTA	Sorteo	El organizador podía reabrir el sorteo en cualquier momento, mover los plazos y volver a sortear hasta que ganara quien él quisiera.	Un solo sorteo por contrato.
F3	MEDIA	Reventa	El organizador podía subir la comisión al 100% justo antes de una compra y quedarse con todo el pago del vendedor.	La comisión queda fijada al publicar y no puede superar el 100%.
F4	MEDIA	Entrada, Reventa	"Renunciar a ser dueño" dejaba la recaudación y las comisiones atrapadas para siempre.	La función de renuncia está deshabilitada.
F5	BAJA	Sorteo	Si alguien depositaba de más, el excedente no se podía recuperar nunca.	Se exige el depósito exacto.
F6	BAJA	Entrada	Con caracteres especiales en la fila o el asiento se podía alterar la imagen y los datos del NFT.	Fila y asiento solo aceptan letras y números.

La **segunda ronda** (9 hallazgos más) tiene su propia tabla en el capítulo 11.

> **Contexto importante: algunas fallas se dejaron a propósito.** Las specs del proyecto son material de un curso. Cada una tiene una sección `[SOLO INSTRUCTOR]` con huecos de seguridad puestos adrede para descubrirlos en clase, como el "sesgo del último revelador" del sorteo. La primera versión de este informe los documentaba como límites de diseño. El profesor pidió corregir en el código todo lo detectado, así que en la segunda ronda se implementó una mitigación para cada uno que la tuviera, con su justificación. Quedan solo dos sin arreglo, porque no lo tienen dentro de un contrato: pagar por fuera de la cadena y el front-running entre compradores (capítulo 12).

Estado de los tests

	Pasan	Fallan	Pendientes (skip a propósito)
Al recibir el código	123	3	12
Después de la primera ronda	135	0	12
Final (dos rondas)	**160**	**0**	**7**

Los pendientes son tests que el curso marcó con `it.skip`. Cinco pasaron a ser tests reales en la segunda ronda; los 7 que quedan son los que no tienen arreglo en el código o no son de seguridad. Las 3 fallas iniciales eran de un archivo de tests local (`Entrada.seguridad.ts`) que usaba una forma de verificación vieja; no eran fallas del contrato.

3. Conceptos que necesitás antes de empezar

Estos son los conceptos que aparecen en los hallazgos. Si ya leíste la guía de smart contracts, algunos te van a sonar.

Gas y el límite por bloque

Cada operación que ejecuta un contrato cuesta **gas**: sumar es barato, guardar un dato es caro. Las transacciones se agrupan en **bloques**, y **cada bloque tiene un tope de gas** (del orden de 30 millones; el valor exacto depende de la red y cambia con el tiempo). Una transacción que necesita más gas que ese tope **no se puede ejecutar nunca**: no importa cuánto pagues.

Analogía: cada viaje tiene un tanque de nafta de tamaño fijo. Si el recorrido necesita más nafta de la que entra en el tanque, el auto no llega nunca, por más plata que tengas.

DoS (denegación de servicio)

En la web, un DoS es "tirar un servidor". En un smart contract es distinto: es lograr que **una función deje de poder ejecutarse**. Pasa típicamente cuando el costo de una función crece con algo que el atacante puede inflar (por ejemplo, la cantidad de participantes) hasta pasar el tope de gas.

Fondos atrapados

Un contrato puede guardar ether. Pero si no existe ninguna función que lo pueda sacar (o la única que existe ya nadie la puede llamar), ese ether queda **atrapado para siempre**. Nadie lo puede recuperar: ni el usuario, ni el organizador, ni el programador. El código no se puede editar después de desplegado.

El mempool y el front-running

Cuando mandás una transacción, no se ejecuta al instante: espera en una sala de espera pública llamada **mempool** hasta que alguien la incluye en un bloque. Cualquiera puede ver lo que hay en el mempool.

QUIÉN DECIDE EL ORDEN: EL QUE ARMA EL BLOQUE

1. En Ethereum sale un bloque nuevo cada unos 12 segundos.
2. Para cada bloque hay un **validador** (o un "constructor de bloques" que trabaja para él) que **elige qué transacciones entran y en qué orden**.
3. Cada transacción ofrece una **propina** para ese validador. En Ethereum se llama *priority fee* y se mide en **gwei** (una milmillonésima de ether).
4. Al validador le conviene ganar lo máximo posible, así que en la práctica **ordena las transacciones de mayor a menor propina**.

> **La idea clave:** no existe la regla de "el que llegó primero va primero". El orden lo decide quien arma el bloque, y lo decide por plata.

Analogía: un cajero de banco que, en vez de atender por orden de llegada, atiende primero a quien le deja la propina más grande. No importa que hayas llegado antes: si alguien que llegó después le ofrece más, pasa adelante. Y como tu formulario está a la vista de todos en la fila, ese alguien sabe exactamente qué estás por hacer.

Front-running es justamente eso: ver la transacción de otro en el mempool y mandar una propia con **más propina**, para que se ejecute **antes** y sacar ventaja. En F3 hay un caso concreto con números.

> **Matices, para no exagerar si te preguntan:**
>
> - **No está garantizado.** Si la transacción de la víctima ya entró en un bloque antes de que el atacante la viera, llega tarde. Tiene pocos segundos, así que en la práctica esto lo hacen programas automáticos (bots), no una persona a mano.
> - **Hay otras vías además de la propina.** Existen servicios para mandar transacciones "privadas" directo a los constructores de bloques, en un orden pactado. Y un validador puede ordenar su propio bloque como quiera. Con la propina alcanza para entender la idea.

El "dueño" del contrato: Ownable

Varios contratos heredan `Ownable` de OpenZeppelin. Les da un **dueño** (`owner()`, en este proyecto el organizador) y un candado (`onlyOwner`) para las funciones que solo él puede usar. Trae dos funciones para cambiar de dueño:

- `transferOwnership(nueva)`: le pasa el control a otra dirección.
- `renounceOwnership()`: deja el contrato **sin dueño para siempre**. El dueño pasa a ser la dirección `0x000…000`, cuya clave no tiene nadie. Es irreversible.

Inyección

Una inyección pasa cuando un dato del usuario se pega dentro de un texto con formato (JSON, HTML, SQL…) sin filtrarlo, y ese dato trae caracteres que **cambian el formato**.

Analogía: un formulario arma la frase Nombre: [lo que escribiste]. Edad: 30. Si en el nombre escribís Juan. Edad: 99. Nota: ignorar lo que sigue, el texto final dice otra cosa de lo que quería el sistema.

Invariante

Una regla que **nunca** debe romperse. Cada spec del proyecto tiene su lista (por ejemplo, el sorteo dice: "nunca se entreguen más entradas que las anunciadas"). Un buen hallazgo de seguridad suele ser "encontré una forma de romper el invariante N".

Sybil

Una persona que se hace pasar por muchas creando muchas direcciones. En una blockchain crear direcciones es gratis, así que **"una dirección" no es lo mismo que "una persona"**.

Entropía: el azar del sorteo convertido en un número

En general, la **entropía** es el grado de impredecibilidad de algo: cuánto azar tiene. Un dado bien tirado tiene mucha entropía (no sabés qué va a salir), un dado cargado tiene poca, y una moneda con dos caras iguales tiene cero.

En el código de Puerta 7 la palabra tiene un uso concreto: **la entropía es el número del que sale el resultado del sorteo**. Funciona como la "semilla" de la que se deriva todo lo demás. Si ese número es impredecible, el sorteo es justo. Si alguien lo puede adivinar o elegir, puede hacer trampa.

DE DÓNDE SALE ESE NÚMERO

En una blockchain no hay un dado: todo es público y determinista. Entonces el sorteo **fabrica** el azar juntando un aporte de cada participante:

1. Cada postulante elige un número secreto (por ejemplo, Ana elige el 11 y Beto el 22).
2. Primero publica un **sobre cerrado** con ese número: el compromiso, que es un hash. Nadie ve el número todavía.
3. Cuando vence el plazo, cada uno **abre su sobre** (revela).
4. El contrato **mezcla** todos los números revelados en uno solo. Ese número es la entropía.

En el código, la mezcla se hace dentro de `revelar()`, una revelación a la vez:

```
_entropia = uint256(keccak256(abi.encodePacked(_entropia, valor, msg.sender)));
```

Paso a paso con Ana y Beto (los valores de hash están acortados):

```
empieza:          _entropia = 0
revela Ana (11):  _entropia = hash(0, 11, Ana)       -> 8f3a...  (un número enorme)
revela Beto (22): _entropia = hash(8f3a..., 22, Beto) -> c71e...  <- entropía final
```

El `keccak256` funciona como una **licuadora**: si cambia **cualquier** ingrediente, aunque sea en un dígito, el resultado sale completamente distinto y no tiene ninguna relación con el anterior. Por eso nadie puede predecir el resultado mirando los valores por separado. Después, `cerrar()` usa esa entropía para "mezclar las cartas" y decidir quién gana (ver F1).

POR QUÉ COMBINAR LOS NÚMEROS DE TODOS

Para que **ningún participante pueda decidir el resultado por su cuenta**. Mientras al menos uno haya elegido su número de verdad al azar, la mezcla final es impredecible para todos los demás.

Para eso sirve el sobre cerrado. Si cada uno publicara su número directamente, el último podría mirar los números de los demás y elegir el suyo justo para ganar. El sobre te obliga a **comprometerte antes de ver** lo que eligieron los otros.

> **Dónde era débil la entropía de este proyecto** (las dos cosas se corrigieron en la segunda ronda, secciones 11.1 y 11.2, con la idea del concepto siguiente):
>
> - **El último en revelar (N2).** Los sobres se abren en público y de a uno. El último ya vio los números de todos y conoce el suyo, así que puede **calcular la entropía final antes de revelar**. Si no le conviene, no revela (y pierde el depósito). Ya no puede elegir *qué* número poner, pero sí *si* lo pone, y eso alcanza para inclinar el resultado.
> - **La compra de entradas (N1).** `Entrada.comprar()` usa como única fuente de azar `block.prevrandao`, un número que trae cada bloque y que es **público** en el momento de la compra. Es azar "a la vista": muy poca entropía real para alguien que la quiera aprovechar.

> **Cómo explicarlo:** "La entropía es el azar del sorteo convertido en un número. Como en la blockchain no hay dados, lo fabricamos mezclando los números secretos de todos los participantes. Mientras uno solo haya elegido el suyo al azar, nadie puede predecir el resultado."

El hash de un bloque futuro: un dato que todavía nadie conoce

Cada bloque tiene una **huella digital** (su hash) que se calcula recién cuando el bloque se arma, a partir de todo lo que contiene. Un contrato puede leer el hash de un bloque ya pasado con `blockhash(número)`. El EVM solo recuerda los **últimos 256**: para bloques más viejos devuelve cero.

Lo útil para la seguridad es el momento: **mientras un bloque no existe, su hash no lo conoce nadie**. Y una vez que existe, queda **fijo para siempre**. Eso permite separar en el tiempo dos cosas que antes ocurrían juntas:

1. **Comprometerse** (pagar, o terminar de revelar) sin conocer el dato que va a decidir el resultado.
2. **Calcular el resultado** cuando el dato ya existe, sin poder cambiarlo.

Analogía: es como pagar la rifa hoy y que el número ganador salga del sorteo de la lotería de mañana. Hoy nadie sabe qué va a salir; mañana sale un número que nadie puede elegir ni deshacer.

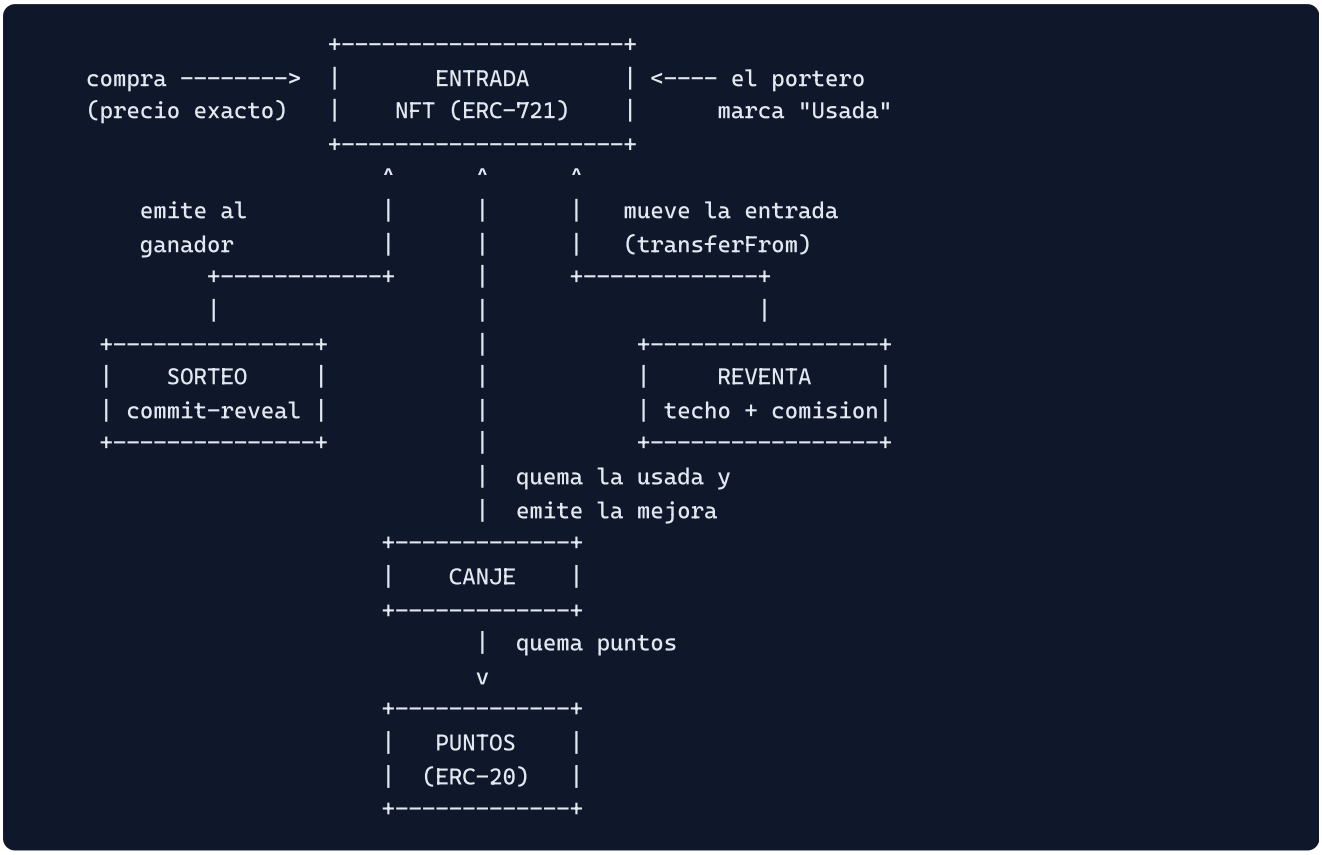
> **El límite:** quien produce ese bloque (un validador) elige qué transacciones pone adentro, y eso cambia el hash. O sea, un validador podría influir en el resultado. Por eso este dato protege contra cualquier comprador o participante, pero no contra alguien que además arme bloques. Para eso haría falta un oráculo de azar externo.

Test de regresión (rojo → verde)

Para cada falla se escribió primero un test que la reproduce. Se lo corrió con el contrato viejo y **falló** (rojo): eso demuestra que la falla existe. Después se corrigió el contrato y el test **pasó** (verde): eso demuestra la corrección. El test queda para siempre en la suite, así que si alguien reintroduce el problema, se entera de inmediato.

4. El sistema en un dibujo: los 5 contratos

Puerta 7 creció: ya no es solo un contrato de entradas. Son cinco contratos que se hablan entre sí.



Contrato	Qué hace	Plata que guarda
Entrada	El NFT de cada entrada. Se compra a precio fijo y recibís un asiento al azar.	Lo recaudado por ventas, hasta que el organizador lo retira.
Sorteo	Rifa entradas entre postulantes con "commit-reveal": primero te comprometés con un valor secreto y después lo revelás.	Los depósitos de los postulantes.
Reventa	Mercado secundario con precio máximo (techo) y una comisión para el organizador.	Lo que les toca a vendedores y organizador, hasta que lo retiran (patrón "pull").
Puntos	Un token (ERC-20) que premia la asistencia.	Nada en ether.
Canje	Quema puntos más una entrada usada y te da una entrada de un sector mejor.	Nada en ether.

> **Permisos cruzados:** `Entrada` tiene una lista de "operadores autorizados" (Sorteo y Canje) que pueden crear entradas para alguien (`emitirPara`) o quemarlas (`quemar`). El organizador los designa después de desplegar. Por eso un error en Sorteo o en Canje termina afectando a Entrada.

Los dos hallazgos más graves (F1 y F2) están en **Sorteo**, que es el contrato que más plata de usuarios custodia y el que tiene la lógica más complicada.

5. F1 · El sorteo se podía trabar para siempre

ALTA | Contrato: `Sorteo.sol` | Función: `cerrar()` | Rompe: invariante 7

Cerrar el sorteo costaba cada vez más a medida que se anotaba gente. Con un evento popular el costo superaba el máximo de un bloque: nadie podía cerrarlo, nadie recibía su entrada y todos los depósitos quedaban congelados para siempre.

CONTEXTO

El sorteo tiene tres etapas: los postulantes **se comprometen** (dejan un depósito), después **revelan** su valor secreto, y al final alguien llama a `cerrar()`. Esa función calcula el resultado y elige los ganadores. Recién después cada uno puede `reclamar()`: el ganador recibe su entrada y el perdedor recupera su depósito.

Lo clave: **sin `cerrar()` no hay `reclamar()`**. Si `cerrar()` no se puede ejecutar, la plata de todos queda adentro.

QUÉ ESTABA MAL

Para elegir **cada** ganador, el código viejo recorría la lista **completa** de postulantes, calculaba un puntaje para cada uno y se quedaba con el menor. Con 100 entradas, recorría la lista entera 100 veces.

ANTES (RESUMIDO)

```
// para cada ganador...
for (uint256 g = 0; g < cantidadGanadores; g++) {
    // ...recorre a TODOS los postulantes
    for (uint256 i = 0; i < _revelados.length; i++) {
        uint256 puntaje = uint256(keccak256(abi.encodePacked(entropia, _revelados[i])));
        if (puntaje < mejor) { mejor = puntaje; mejorIndice = i; }
    }
    _esGanador[_revelados[mejorIndice]] = true;
}
```

Además, antes de esos bucles había otro que recorría a todos para calcular la entropía. El costo total crecía como **ganadores × postulantes**: si se duplican las dos cosas, el costo se multiplica por cuatro.

Analogía: una maestra tiene que elegir 100 alumnos premiados entre 300. En vez de mezclar los nombres una vez, para cada premio relee la lista completa de 300. Son 30.000 lecturas, y si la escuela crece, el trabajo crece mucho más rápido que la escuela.

CÓMO SE EXPLOTABA

Medimos el costo real de `cerrar()` en el código viejo:

Postulantes	Entradas	Gas de `cerrar()` (antes)
3	1	≈ 101.000
18	1	205.030
18	9	523.992
18	17	782.249

Cada postulante extra sumaba unos 7.000 gas, y cada ganador extra, entre 32.000 y 40.000 (y ese número sube con cada postulante). Extrapolando: **con 100 entradas, a partir de unos 280 postulantes, `cerrar()` supera los 30 millones de gas**. Hay dos caminos para llegar ahí:

Escenario A: sin atacante. Un recital con 100 entradas sorteadas y 300 interesados. Es justo el caso para el que existe el sorteo ("más demanda que aforo", dice la spec).

Escenario B: con atacante (griefing). Alguien quiere arruinar el sorteo:

1. Crea cientos de direcciones (es gratis; ver "Sybil" en el cap. 3).
2. Se postula desde todas, pagando un depósito en cada una, y las revela.
3. La lista de revelados crece hasta que `cerrar()` no entra en un bloque.
4. Nadie puede cerrar el sorteo. Nadie puede reclamar. **Los depósitos de todos los participantes reales quedan atrapados.**

> **Impacto:** rompe el invariante 7 de la spec ("un postulante que reveló correctamente y perdió no se puede quedar sin su depósito"). La plata queda congelada para siempre: el contrato no se puede editar y no hay ninguna otra función que la saque.

LA CORRECCIÓN

Se hicieron dos cambios que juntos eliminan la dependencia de la cantidad de postulantes.

Cambio 1: la entropía se acumula al revelar. (Qué es la entropía y cómo se arma: ver el capítulo 3.) En vez de recorrer a todos al final para combinar sus valores, cada postulante suma su valor en el momento en que revela. El resultado es **idéntico** (misma fórmula, mismo orden), pero el trabajo se reparte: cada uno paga su parte y `cerrar()` ya no recorre a nadie.

DESPUÉS: EN REVELAR()

```
// Se acumula acá (en orden de revelación, igual que antes al
// cerrar) para que `cerrar` no recorra a todos los postulantes.
_entropia = uint256(keccak256(abi.encodePacked(_entropia, valor, msg.sender)));
```

Cambio 2: los ganadores se eligen con "Fisher-Yates parcial". Es la forma clásica de mezclar una lista al azar. Se mezclan solo las primeras posiciones (tantas como entradas haya) y esos son los ganadores.

Analogía con un mazo de cartas: para sacar 3 ganadores de 300 cartas, sacás una carta al azar de todo el mazo y la ponés primera. Sacás otra al azar entre las 299 que quedan y la ponés segunda. Sacás una

tercera. Listo: hiciste 3 pasos, no 300, y cada carta tuvo la misma chance.

DESPUÉS: EN CERRAR()

```
uint256 total = _revelados.length;
uint256 cantidadGanadores = cantidadEntradas < total ? cantidadEntradas : total;
// Si alcanzan para todos no hay nada que sortear
if (cantidadGanadores < total) {
    for (uint256 g = 0; g < cantidadGanadores; g++) {        // un paso por ENTRADA
        uint256 j = g + (uint256(keccak256(abi.encodePacked(entropia, g))) % (total - g));
        address ganador = _revelados[j];                     // carta al azar
        _revelados[j] = _revelados[g];                       // intercambio
        _revelados[g] = ganador;
        _esGanador[ganador] = true;
    }
}
```

Y un detalle extra: si hay entradas para todos, no se sortea nada. `reclamar()` trata a todo revelado como ganador sin marcarlo uno por uno:

```
bool gano = cantidadEntradas >= _revelados.length || _esGanador[msg.sender];
```

RESULTADO MEDIDO

En la segunda ronda el cierre pasó a tener dos pasos (ver N2, sección 11.2). El primero solo fija un bloque y cuesta siempre **49.428 gas**. La tabla compara el cierre viejo con el segundo paso del nuevo, que es el que sortea:

Postulantes	Entradas	Antes	Después (paso 2)
3	1	101.030	**111.137**
18	1	205.030	**118.737**
18	9	523.992	369.579
18	17	782.249	590.424

La prueba de que funciona: con 1 entrada, pasar de 3 a 18 postulantes antes sumaba **104.000 gas**; ahora suma unos pocos miles. Esa diferencia no crece con más postulantes: depende de si la posición sorteada cae sobre la misma casilla, que hace el intercambio más barato. Agregar participantes ya no encarece el cierre, así que un atacante no tiene nada que inflar.

Límite que queda: cada entrada sorteada cuesta entre 28.000 y 31.000 gas. La diferencia con antes es quién controla ese número: ahora lo fija el organizador al abrir el sorteo, no un atacante. En la segunda ronda se le puso además un tope de 500 entradas (R2, sección 11.6), con el que el cierre usa como mucho la mitad de un bloque.

> **¿Cambian los ganadores?** Sí: con la misma entropía, el algoritmo nuevo elige ganadores distintos que el viejo. No es un problema. La spec solo pide "ordenar con la entropía y tomar los primeros", que es exactamente lo que hace Fisher-Yates. Ningún test fijaba quién gana (lo aclaran explícitamente), y el front no recalcula ganadores por su cuenta.

CÓMO LO VERIFICAMOS

- **Test "el costo de cerrar no crece con la cantidad de postulantes (sin DoS)":** mide el gas de `cerrar()` con 3 y con 18 postulantes y exige que la diferencia sea menor a 15.000. Con el código viejo falló (diferencia: **104.000**). Ahora la diferencia es de 7.600 (en otras corridas, 0) y no aumenta con más postulantes.
- **Test "con más postulantes que entradas ganan exactamente las anunciadas, una por cabeza":** 10 postulantes, 3 entradas. Verifica que haya exactamente 3 ganadores, nadie con más de una entrada, y que los 7 perdedores recuperen su depósito. Es la red de seguridad del algoritmo nuevo.

> **Cómo explicarlo:** "Para elegir cada ganador, el contrato releía la lista completa de participantes. El costo crecía tan rápido que, con unos cientos de personas, cerrar el sorteo se volvía imposible y la plata de todos quedaba congelada. Lo cambiamos por una mezcla clásica de cartas, Fisher-Yates, que hace un paso por entrada. Ahora el costo depende de cuántas entradas se rifan, no de cuánta gente se anota."

6. F2 · El sorteo se podía reabrir

ALTA | Contrato: `Sorteo.sol` | Función: `abrirSorteo()` | Rompe: invariantes 2, 3 y 5

El organizador podía reabrir el sorteo cuando quisiera: mover los plazos en el medio y, después del cierre, volver a sortear las veces que hiciera falta hasta que ganara quien él quería, entregando además más entradas de las anunciadas.

CONTEXTO

El objetivo central del sorteo, según la spec, es repartir **"sin que nadie pueda predecir el resultado ni inclinarlo a su favor"**. "Nadie" incluye al organizador: el commit-reveal existe justamente para no depender de su buena fe.

El contrato guarda en qué etapa está con dos marcas (`abierto` y `cerrado`) y dos fechas límite (`finCompromiso` y `finRevelacion`).

QUÉ ESTABA MAL

ANTES

```
function abrirSorteo(uint256 cantidadEntradas_, uint256 duracionCompromiso, uint256
duracionRevelacion)
    external onlyOwner
{
    cantidadEntradas = cantidadEntradas_;                    // se puede cambiar
    finCompromiso = block.timestamp + duracionCompromiso;    // plazos nuevos
    finRevelacion = finCompromiso + duracionRevelacion;
    abierto = true;
    cerrado = false;                                         // "des-cierra" el sorteo
    emit SorteoAbierto(cantidadEntradas, finCompromiso, finRevelacion);
}
```

No había ningún control de si el sorteo ya estaba en curso o ya había terminado. Se podía llamar en cualquier momento, y reiniciaba plazos y estado **sin borrar** postulaciones, revelaciones ni ganadores anteriores.

CÓMO SE EXPLOTABA

Ataque 1: volver a sortear hasta que gane mi amigo. Sorteo de 1 entrada entre Ana, Beto y otros:

1. Se cierra el sorteo y gana Ana. Ana reclama su entrada.
2. El organizador quería que ganara Beto. Llama a `abrirSorteo` otra vez: `cerrado` vuelve a `false` y hay plazos nuevos.
3. Desde una dirección propia se postula y revela un valor nuevo. Eso cambia la entropía.
4. Cierra de nuevo. Si no ganó Beto, repite desde el paso 2.
5. Cuando gana Beto, Beto reclama. Ana conserva la suya (la marca de ganadora no se borra). **Se entregaron 2 entradas en un sorteo de 1.**

Ataque 2: mover los plazos en el medio. Durante la etapa de revelación, el organizador reabre. Las revelaciones quedan bloqueadas y se vuelven a aceptar compromisos nuevos, después de que ya había vencido el plazo original.

> **Impacto:** rompe el invariante 5 ("no se entregan más entradas que las anunciadas"), el 2 ("no se acepta un compromiso después del plazo") y el 3. Sobre todo, rompe la promesa central del sorteo: el resultado se puede inclinar.

LA CORRECCIÓN

Un sorteo por contrato. Si ya se abrió una vez, no se puede volver a abrir.

DESPUÉS

```
function abrirSorteo(...) external onlyOwner {
    // Un solo sorteo por contrato: reabrir movería los plazos de la
    // ronda en curso (invariantes 2 y 3) o, ya cerrada, recalcularía
    // ganadores sobre reclamos hechos (invariante 5).
    if (abierto) revert SorteoYaAbierto();
    cantidadEntradas = cantidadEntradas_;
    finCompromiso = block.timestamp + duracionCompromiso;
    finRevelacion = finCompromiso + duracionRevelacion;
    abierto = true;
    cerrado = false;   // ya no hace falta: nunca puede estar en true acá
    emit SorteoAbierto(cantidadEntradas, finCompromiso, finRevelacion);
}
```

> **¿Por qué un sorteo por contrato y no "limpiar todo y reabrir"?** Porque es mucho más simple y seguro, y coincide con la spec, que deja "sorteos simultáneos para varios eventos" fuera de alcance. Si se necesita otro sorteo, se despliega otro contrato `Sorteo`: es barato y cada uno tiene su historia limpia.

CÓMO LO VERIFICAMOS

Test "no se puede reabrir: ni con el sorteo en curso ni después del cierre": intenta reabrir durante la revelación y después del cierre. En los dos casos exige el error `SorteoYaAbierto`. Con el código viejo falló (reabría sin problema).

> **Cómo explicarlo:** "El sorteo se podía reabrir cuantas veces se quisiera, y cada reapertura permitía volver a sortear sin borrar a los ganadores anteriores. O sea: el organizador podía tirar los dados hasta que salieran como él quería. Ahora un contrato hace un solo sorteo."

7. F3 · La comisión de reventa se podía cambiar a último momento

MEDIA | Contrato: `Reventa.sol` | Funciones: `configurar()`, `comprar()` | Técnica: front-running

El organizador podía ver una compra pendiente, subir la comisión al 100% justo antes de que se ejecutara y quedarse con todo el pago, dejando al vendedor sin cobrar nada.

CONTEXTO

En la reventa, un vendedor publica (**lista**) su entrada a un precio. Cuando alguien la compra, el pago se divide: una **comisión** para el organizador (por ejemplo, 10%) y el resto para el vendedor. Ninguno de los dos recibe la plata en el momento: queda anotada y cada uno la retira después (patrón "pull", que protege contra reentradas).

El vendedor publica **sabiendo** cuál es la comisión. Si es 10% y vende a 0,1 ETH, espera cobrar 0,09 ETH.

QUÉ ESTABA MAL

La comisión no se guardaba en el listado: se leía de la configuración general **en el momento de la compra**.

ANTES

```
function comprar(uint256 idListado) external payable {
    ...
    uint256 comision = (listado.precio * comisionPorcentaje) / 100;  // valor GLOBAL, del
momento
    _retirable[listado.vendedor] += listado.precio - comision;
    _retirable[owner()] += comision;
    ...
}

function configurar(uint256 techoPorcentaje_, uint256 comisionPorcentaje_) external
onlyOwner {
    techoPorcentaje = techoPorcentaje_;
    comisionPorcentaje = comisionPorcentaje_;        // sin ningún límite
}
```

CÓMO SE EXPLOTABA

Momento	Qué pasa	Propina ofrecida
Antes	Comisión configurada: 10%. El vendedor lista su entrada a 0,1 ETH. Espera cobrar 0,09.	—
0 s	Un comprador manda `comprar()` pagando 0,1 ETH. La transacción queda esperando en el **mempool**, a la vista de todos.	1 gwei
3 s	El organizador (o un bot suyo) la ve y manda `configurar(110, 100)`, es decir, comisión del 100%.	**5 gwei**
12 s	El validador arma el bloque y **ordena por propina**: primero `configurar` (5 gwei), después `comprar` (1 gwei).	—
Resultado	La compra se procesa con 100% de comisión: el organizador recibe **0,1 ETH** y el vendedor **0**. El comprador recibe su entrada, así que de su lado nadie ve nada raro.	—

Los segundos y las propinas son ilustrativos. Lo que importa es la relación: el organizador llega después pero ofrece más.

> **¿Por qué se ejecuta antes la del organizador, si la mandó después?** Porque dentro de un bloque el orden no es por llegada: lo decide el validador que arma el bloque, y él cobra las propinas. Ordenar de mayor a menor propina es lo que más le conviene, así que la transacción que ofrece 5 gwei queda delante de la que ofrece 1, sin importar cuál llegó primero. (Más detalle en "El mempool y el front-running", cap. 3.)

Qué reprodujo el test: el test no simula el mempool ni las propinas. Ejecuta `configurar` y después `comprar`, en ese orden, que es **el efecto** de que el organizador gane la carrera. Lo que demuestra es la consecuencia: con el código viejo, el evento `Comprado` registró una comisión de **0,1 ETH, el pago entero**, y el vendedor cobró 0.

Problema extra: no había tope. Con una comisión de 101%, la cuenta `precio - comisión` da negativo. Solidity 0.8 cancela (revierte) cualquier resta que dé negativa, así que **todas las compras fallaban** y el mercado quedaba bloqueado.

> **Impacto:** robo del pago del vendedor por parte de quien controla la configuración, o mercado bloqueado por un valor mal cargado. Es severidad media porque requiere que actúe el organizador, aunque la regla de la spec ("el resto va al vendedor") se puede romper.

LA CORRECCIÓN

La comisión se **congela en el listado** al momento de publicar: es la que el vendedor aceptó. Cambiar la configuración después solo afecta a los listados nuevos. Además, no se puede configurar más de 100%.

DESPUÉS

```
struct DatosListado {
    uint256 tokenId;
    address vendedor;
    uint256 precio;
    uint256 comisionPorcentaje;   // fijada al listar: la que el vendedor aceptó
    bool activo;
}

function configurar(...) external onlyOwner {
    if (comisionPorcentaje_ > 100) revert ComisionInvalida();
    ...
}

// en listar():   comisionPorcentaje: comisionPorcentaje,   // se copia la vigente
// en comprar():  uint256 comision = (listado.precio * listado.comisionPorcentaje) / 100;
```

La corrección vuelve inútil la carrera: aunque el organizador siga pasando primero con más propina, la compra ya no lee la configuración del momento. Usa la comisión grabada en el listado, la que aceptó el vendedor. Ganar la carrera ya no le da nada.

Principio general: las condiciones de un acuerdo se fijan cuando las dos partes se comprometen, no cuando se cobra. Es el mismo principio por el que un ticket de compra guarda el precio del momento y no el precio de hoy.

CÓMO LO VERIFICAMOS

- **Test "la comisión queda fijada al listar: subirla después no recorta al vendedor":** lista con 10%, sube la comisión a 100% y compra. Exige que la comisión registrada sea la del 10% y que el vendedor retire 0,09 ETH. Con el código viejo falló (registró 0,1 ETH de comisión).
- **Test "revierte si la comisión supera el 100%":** exige el error `ComisionInvalida`.

Cómo explicarlo: "La comisión se leía en el momento de pagar, no en el de publicar. El organizador podía ver una compra en la sala de espera pública, subir la comisión al 100% y pasar adelante pagando una propina. El vendedor terminaba vendiendo gratis. Ahora la comisión queda grabada en la publicación."

8. F4 · "Renunciar" dejaba la plata atrapada

MEDIA | Contratos: `Entrada.sol`, `Reventa.sol` | Función: `renounceOwnership()` (heredada)

Rompe: invariante 8 de Reventa

> **Una función heredada permitía que el organizador renunciara a ser dueño. Un solo clic equivocado dejaba para siempre la recaudación de entradas y las comisiones de reventa sin nadie que las pudiera retirar.**

CONTEXTO

Los contratos heredan `Ownable` de OpenZeppelin, que trae `renounceOwnership()`. Es útil en otros proyectos ("quiero que nadie controle más este contrato"), pero acá el dueño es **la única persona que puede retirar la plata**.

QUÉ ESTABA MAL

En `Entrada`, retirar exige ser el dueño:

```
modifier soloOrganizador() {
    if (msg.sender != owner()) revert("Solo el organizador");
    _;
}
function retirar() external soloOrganizador {
    (bool ok, ) = owner().call{value: address(this).balance}("");
    ...
}
```

Después de renunciar, `owner()` pasa a ser `0x000…000`. Para llamar a `retirar()` habría que firmar como esa dirección, y **nadie tiene su clave**.

En `Reventa` es peor: cada venta sigue acreditando la comisión a `owner()`, es decir a `0x000…000`. Esa plata se sigue acumulando en una cuenta que nadie puede usar.

```
_retirable[owner()] += comision;    // con owner() = 0x0, se acumula para nadie
```

CÓMO SE EXPLOTABA

No hace falta un atacante sofisticado:

- **Error humano:** un script de administración o un botón mal programado llama a `renounceOwnership` en vez de `transferOwnership`.
- **Llave robada:** alguien que roba la llave del organizador y no puede llevarse la plata (por ejemplo, porque se retiró hace poco) igual puede hacer daño irreparable.

En los dos casos: **toda la recaudación de entradas queda bloqueada para siempre**, no se pueden emitir más entradas, y en Reventa las comisiones de todas las ventas futuras se pierden.

> **Impacto:** pérdida total e irreversible de fondos. Rompe el invariante 8 de Reventa ("nunca queden fondos atrapados sin que nadie pueda retirarlos"). Es severidad media porque necesita una acción del organizador.

LA CORRECCIÓN

Se reemplazó la función heredada por una que siempre revierte:

DESPUÉS (ENTRADA)

```
/// Deshabilitada: `retirar` paga a `owner()`, así que sin organizador lo
/// recaudado quedaría atrapado. Para cambiar de organizador: transferOwnership.
function renounceOwnership() public pure override {
    revert("Renuncia deshabilitada");
}
```

DESPUÉS (REVENTA)

```
function renounceOwnership() public pure override {
    revert RenunciaDeshabilitada();
}
```

> **¿Por qué un mensaje distinto en cada contrato?** Por convención del proyecto: `Entrada` usa mensajes de texto (sus tests verifican el texto literal), y `Reventa` usa errores con nombre (`error Foo()`), que es lo que pide su spec.

> **¿Y Sorteo, Canje y Puntos?** En la primera ronda no se tocaron porque no guardaban plata para el dueño. En la segunda, el **Sorteo** empezó a guardarla (ver R1, sección 11.4), así que también se le deshabilitó la renuncia. En Canje y Puntos renunciar solo hace perder la administración, no fondos.

Cambiar de organizador sigue siendo posible con `transferOwnership`. En la segunda ronda ese traspaso pasó a ser en dos pasos, para que un error de tipeo en la dirección nueva no pierda el control (R3, sección 11.5).

CÓMO LO VERIFICAMOS

- **Entrada, test "renounceOwnership revierte":** después de una venta intenta renunciar, exige el error, verifica que el dueño sigue siendo el organizador y que puede retirar.
- **Reventa, test "renounceOwnership revierte":** exige el error `RenunciaDeshabilitada`.
- El test local H1 de `Entrada.seguridad.ts` antes "documentaba" el problema (afirmaba que los fondos quedaban bloqueados). Se actualizó para afirmar la corrección.

> **Cómo explicarlo:** "OpenZeppelin trae una función para que el dueño renuncie. Pero acá el dueño es el único que puede retirar la plata: si renuncia, la plata queda en una caja fuerte sin llave. La deshabilitamos. Para cambiar de dueño se sigue pudiendo transferir."

9. F5 · El depósito de más quedaba atrapado

BAJA | Contrato: `Sorteo.sol` | Función: `comprometer()`

El sorteo aceptaba depósitos mayores al pedido, pero al devolver solo devuelve el monto exacto. Lo que sobraba quedaba adentro para siempre.

QUÉ ESTABA MAL

```
// en comprometer():
if (msg.value < deposito) revert DepositoInsuficiente();   // acepta más

// en reclamar(), para el que perdió:
msg.sender.call{value: deposito}("");                      // devuelve solo lo justo
```

CÓMO SE EXPLOTABA

No es un ataque, es una pérdida por error. Por ejemplo, el depósito es 0,01 ETH y un usuario manda 0,1 por un cero de más (o un front con un bug manda de más). Si pierde, recupera 0,01: **los 0,09 restantes no se recuperan nunca**, porque no existe ninguna función que los saque.

LA CORRECCIÓN

```
if (msg.value < deposito) revert DepositoInsuficiente();
// `reclamar` devuelve exactamente `deposito`: un excedente no
// tendría por dónde salir del contrato.
if (msg.value > deposito) revert DepositoExcedido();
```

Se agregó un error nuevo (`DepositoExcedido`) en vez de reusar `DepositoInsuficiente`, para que el mensaje no confunda ("insuficiente" cuando en realidad mandaste de más). Es el mismo criterio de "pago exacto" que ya usa `Entrada.comprar`.

CÓMO LO VERIFICAMOS

Test "revierte si el depósito excede lo requerido": manda 1 wei de más y exige el error `DepositoExcedido`. Con el código viejo falló (el depósito se aceptaba).

> **Cómo explicarlo:** "El sorteo aceptaba que pagaras de más, pero solo te devolvía lo justo. El sobrante quedaba atrapado. Ahora, como en la compra de entradas, se exige el monto exacto."

10. F6 · Inyección en la imagen y los datos del NFT

BAJA | Contrato: `Entrada.sol` | Funciones: `emitir()`, `tokenURI()` | Técnica: inyección

La fila y el asiento se pegaban sin filtrar dentro de los datos y de la imagen del NFT. Con caracteres especiales, quien emite podía cambiar lo que muestra la entrada, por ejemplo dibujarle "VIP" a una entrada común.

CONTEXTO

Cada entrada tiene una "ficha" (metadata) que devuelve `tokenURI()`. Es un texto en formato **JSON** con el nombre y una imagen. La imagen es un **SVG**, que es un dibujo escrito como texto. Las wallets y los marketplaces leen ese JSON para mostrar el NFT. El contrato lo arma **pegando textos**:

```
json = '{"name":"Entrada ' + sector + '-' + fila + '-' + asiento + '", ... ,"image":"' +
imagen + '"}'
svg  = '... <text ...>' + fila + '-' + asiento + '</text> ...'
```

Con una fila normal como `A`, sale `{"name":"Entrada 1-A-1", ...}`. Todo bien.

QUÉ ESTABA MAL

La única validación era "no vacío y distinto de `0`". Se aceptaba cualquier carácter, incluidos los que tienen significado especial en JSON (`"`) y en SVG (`<`, `>`).

```
if (b.length == 0 || (b.length == 1 && b[0] == "0")) revert("Fila invalida");
// cualquier otra cosa pasa: comillas, <, >, ...
```

CÓMO SE EXPLOTABA

Ejemplo 1: alterar el JSON. Se emite una entrada con la fila:

```
A","image":"https://sitio-falso.com/x.png
```

La comilla "cierra" el texto del nombre antes de tiempo y lo que sigue se interpreta como un campo nuevo. El JSON queda con **dos** campos `image`. Según el programa que lo lea, se muestra la imagen del atacante (algunos se quedan con el primer campo repetido) o el JSON queda inválido y la entrada se ve rota.

Ejemplo 2: alterar el dibujo. Con una fila como `</text><text x="20" y="190">VIP</text><text>`, el SVG pasa a tener un texto extra "VIP" dibujado en la entrada. Con `<script>` se puede meter código, que algunos visores ejecutan si muestran el SVG directamente en la página.

> **¿Por qué severidad baja?** Solo pueden poner fila y asiento quien emite: el organizador (`emitir`) y los operadores (`emitirPara`). `Canje` copia los datos de una entrada existente y `Sorteo` usa la letra `S` más una dirección en hexadecimal, así que hoy ninguno es un camino de ataque. El riesgo real es una llave del organizador robada, o un error de carga (por ejemplo, un CSV con comillas que rompe la metadata de cientos de entradas).

LA CORRECCIÓN

Fila y asiento solo pueden tener **letras y números** (`0-9`, `A-Z`, `a-z`). La spec ya lo decía: "fila y asiento pueden ser alfanuméricas".

DESPUÉS

```
function _validarFila(string memory fila) private pure {
    bytes memory b = bytes(fila);
    if (b.length == 0 || (b.length == 1 && b[0] == "0") || !_esAlfanumerico(b)) {
        revert("Fila invalida");
    }
}
// _validarAsiento: igual

/// Solo [0-9A-Za-z], como define la spec.
function _esAlfanumerico(bytes memory b) private pure returns (bool) {
    for (uint256 i = 0; i < b.length; i++) {
        bytes1 c = b[i];
        bool ok = (c >= "0" && c <= "9") ||
            (c >= "A" && c <= "Z") ||
            (c >= "a" && c <= "z");
        if (!ok) return false;
    }
    return true;
}
```

> **¿Por qué una "lista blanca" y no "escapar" los caracteres?** Escapar significa transformar `"` en `\"`, y OpenZeppelin tiene una función para eso en JSON. Pero acá el mismo dato va a dos formatos distintos (JSON y SVG), con reglas de escape distintas: es fácil hacerlo bien en uno y olvidarse del otro. La lista blanca ("solo se permiten estos caracteres") es más simple, cubre los dos formatos a la vez y además es lo que pide la spec.

La validación se aplica en los dos caminos de emisión (`emitir` y `emitirPara`), porque los dos usan las mismas funciones. El asiento que genera el Sorteo (`0x` seguido de 40 caracteres hexadecimales) es alfanumérico, así que sigue funcionando.

CÓMO LO VERIFICAMOS

- **Test "revierte si la fila o el asiento traen caracteres no alfanuméricos":** prueba la fila `A","image":"https://x` y el asiento `1</text><script>`. Exige "Fila invalida" y "Asiento invalido". Con el código viejo falló (emitió sin problemas).

- El test local H3 de `Entrada.seguridad.ts` antes afirmaba que "una comilla rompe el JSON". Se actualizó para afirmar que la comilla se rechaza al emitir.
- Todos los tests que ya existían (que usan filas como `AB`, `C12`, `10B`) siguen pasando.

> **Cómo explicarlo:** "La imagen de la entrada se arma pegando textos. Si en la fila ponías una comilla o un signo menor, podías 'romper' ese texto y meterle cosas, como un 'VIP' falso dibujado en la entrada. Ahora fila y asiento solo aceptan letras y números, que es lo que decía la spec desde el principio."

11. Segunda ronda: las correcciones que pidió el profesor

La primera versión de este informe corregía 6 fallas y dejaba otras documentadas como "límites de diseño" o "recomendaciones". El profesor pidió dos cosas:

1. **Que lo detectado se corrija en el código**, no que solo se describa.
2. **Evidencia de que los contratos se ejecutaron**, con capturas.

Este capítulo cubre lo primero; el capítulo 13 cubre lo segundo. Se corrigieron **9 hallazgos más**. Dos de ellos cambian cómo se usa el sistema (la compra y el cierre del sorteo pasan a tener dos pasos), así que también se actualizaron las specs (cada una tiene una sección nueva, "Cambios tras la revisión de seguridad"), el front y los scripts.

#	Severidad	Contrato	Problema	Corrección
N1	MEDIA	Entrada	Un contrato podía comprar y revertir hasta conseguir el asiento que quería.	Compra en dos pasos: el asiento sale del hash de un bloque posterior al pago.
N2	MEDIA	Sorteo	El último en revelar podía calcular el resultado antes de decidir si revelaba.	Cierre en dos pasos: se suma el hash de un bloque fijado después de la revelación.
N3	MEDIA	Entrada	El portero podía marcar entradas en cualquier momento, incluso el día antes.	Ventana de ingreso: solo durante el horario del evento.
R1	MEDIA	Sorteo	Los depósitos que no se devuelven quedaban atrapados para siempre.	`retirar()` para el organizador, reservando lo que se les debe a los perdedores.
R3	MEDIA	Los 5	Un traspaso a una dirección mal tipeada perdía el control (y la plata) para siempre.	Traspaso en dos pasos (`Ownable2Step`).
R2	BAJA	Sorteo	Se podía anunciar un sorteo tan grande que nunca entrara en un cierre.	Tope de 500 entradas.
R4	BAJA	Entrada	Un operador podía quemar cualquier entrada sin permiso de su dueño.	`quemar` exige el permiso del dueño.
R5	BAJA	Reventa	Un listado viejo revivía si la entrada se iba y volvía al vendedor.	El listado deja de valer si la entrada cambió de dueño.
N6	BAJA	Canje	El canje verificaba la autorización de puntos pero no la descontaba.	Los puntos pasan por `transferFrom`, que la consume.

> **¿Y los huecos deliberados del curso?** N1, N2, N3 y N6 son huecos que las specs dejaron a propósito para descubrirlos en clase. Para el sesgo del último revelador (N2), la spec del instructor pide que cada pareja **elija una mitigación y la justifique por escrito**; esa justificación está en la sección 11.2. Dos huecos siguen sin solución porque no la tienen dentro de un contrato (pagar por fuera y front-running, capítulo 12).

11.1 N1 · Comprar y revertir hasta elegir el asiento

MEDIA | Contrato: `Entrada.sol` | Funciones: `comprar()`, `confirmarCompras()` (nueva)

Rompe: "el comprador no elige asiento"

> **La compra elegía el asiento con un número público en el mismo momento del pago. Un contrato atacante podía comprar, mirar qué le tocó y deshacer todo si no le gustaba, hasta quedarse con el asiento que quería.**

CONTEXTO

La spec decide que el comprador **no elige** su asiento: le toca uno al azar, para que un revendedor no pueda acaparar los mejores lugares.

QUÉ ESTABA MAL

ANTES

```
function comprar() external payable {
    ...
    uint256 indice = uint256(block.prevrandao) % _disponibles.length;   // número público
    uint256 tokenId = _disponibles[indice];
    ...
    _update(msg.sender, tokenId, address(0));   // la entrega, en la MISMA transacción
}
```

Había dos problemas juntos:

1. `block.prevrandao` es un número del bloque que **ya es público** cuando se ejecuta la compra.
2. La entrada se entregaba **en la misma transacción** que el pago. Por la regla del todo-o-nada, quien compra desde un contrato puede mirar el resultado antes de que termine la transacción y, si no le gusta, **revertir todo**: la compra y el pago se deshacen y solo pierde el gas.

CÓMO SE EXPLOTABA

Con un contrato como este (está en el repo, `contracts/test/AtacanteCompra.sol`, solo para tests):

```
function atacar(uint256 objetivo) external payable {
    entrada.comprar{value: msg.value}();
    if (entrada.ownerOf(objetivo) != address(this)) {
        revert("No toco el asiento elegido");   // deshace la compra y el pago
    }
}
```

El atacante lo llama una vez por bloque hasta que le sale. En la demo contra el código subido, **consiguió la entrada #3 en el intento 3** (en otra corrida, en el 4). Cada intento fallido le costó solo el gas.

LA CORRECCIÓN: LA COMPRA EN DOS PASOS

DESPUÉS · PASO 1: PAGAR RESERVA

```
function comprar() external payable {
    if (msg.value != precio) revert("Pago incorrecto");
    // Las reservas pendientes ya tienen su entrada apartada.
    if (_disponibles.length <= _reservasCreadas - _reservasProcesadas) revert("Sin entradas
disponibles");
    uint256 idReserva = ++_reservasCreadas;
    _reservas[idReserva] = Reserva({ comprador: msg.sender, bloque: block.number });
    emit CompraReservada(idReserva, msg.sender);
}
```

DESPUÉS · PASO 2: ASIGNAR (CUALQUIERA PUEDE LLAMARLA)

```
function confirmarCompras(uint256 maximo) external {
    // hasta `maximo` reservas, EN ORDEN DE LLEGADA
    ...
        if (block.number <= reserva.bloque + 1) return;          // el hash todavía no
existe
        bytes32 semilla = blockhash(reserva.bloque + 1);         // hash del bloque
SIGUIENTE al pago
        if (semilla == bytes32(0)) {                               // pasaron más de 256
bloques
            reserva.bloque = block.number;  emit CompraReprogramada(idReserva);  return;
        }
        uint256 indice = uint256(keccak256(abi.encodePacked(semilla, idReserva))) %
_disponibles.length;
        ... // swap-and-pop, estado Vendida, entrega y evento EntradaComprada
}
```

Por qué funciona:

1. **Cuando se paga, el número que elige el asiento todavía no existe.** Es el hash del bloque siguiente, y el bloque siguiente todavía no se armó (ver "El hash de un bloque futuro", cap. 3). No hay nada que mirar.
2. **Cuando ese número existe, ya está fijo.** Revertir la confirmación no lo cambia: el próximo intento da exactamente el mismo asiento. "Simular y revertir" deja de servir.
3. **Las reservas se procesan en orden de llegada** y cualquiera puede confirmarlas. Si cada uno eligiera cuándo confirmar la suya, podría esperar a que la lista de disponibles cambiara (porque compró otro) hasta que el mismo número le diera un asiento mejor. Con el orden fijo, la lista en el momento de asignar cada reserva no depende de nadie.
4. **Si nadie confirma en 256 bloques** (unos 51 minutos), el EVM ya no recuerda ese hash y la reserva toma un bloque nuevo.

Analogía: pagás la entrada y el número de asiento lo saca una bolilla que se sortea después. Cuando pagás no sabés cuál va a ser, y cuando sale ya no podés devolver la entrada para probar suerte de nuevo.

Límites que quedan (para no exagerar):

- Quien produce ese bloque (un validador) podría influir en su hash. Para eso hay que ser validador, no solo comprador: la barrera sube muchísimo.
- Un atacante que ve un hash desfavorable podría esperar que nadie confirme durante 256 bloques para que se reprograme. No lo controla: cualquiera puede confirmar (los compradores que vienen atrás, o un bot del organizador).
- Si el organizador emite un lote nuevo mientras hay reservas pendientes, la lista cambia. Es un caso raro (se emite antes de vender) y el comprador no lo controla.

Qué cambia para el usuario: en el front, "Comprar entrada" y después "Recibir entrada". `disponibles()` ya descuenta las reservas pendientes.

CÓMO LO VERIFICAMOS

- **"Pagar solo reserva":** después de pagar el comprador no tiene entrada y `disponibles()` bajó; en el bloque siguiente todavía no se puede asignar; uno más tarde, sí.
- **"Un contrato atacante no puede comprar y revertir…":** diez intentos en bloques distintos. Con el código viejo **falló** (el atacante consiguió la #3); ahora ninguno lo logra.
- **"El asiento sale del hash del bloque siguiente…":** confirma otra cuenta, cinco bloques después, y el asiento es exactamente el que se calcula con ese hash. No depende de quién ni de cuándo.
- **Orden de llegada** y **reprogramación a los 256 bloques**: un test cada uno.
- **Demo N1:** antes "consiguió la entrada #3 en el intento 3", después "10 intentos, ninguno le dio la entrada #3".

Cómo explicarlo: "Antes el asiento se sorteaba con un número que ya era público en el momento de pagar, y en la misma transacción. Un bot podía comprar, mirar el asiento y deshacer todo si no le gustaba. Ahora pagás y reservás, y el asiento sale del hash del bloque siguiente, que no existe cuando pagás. Cuando existe, ya está fijo: deshacer no sirve."

11.2 N2 · El último en revelar podía calcular el resultado

MEDIA | Contrato: `Sorteo.sol` | Función: `cerrar()` | Rompe: invariante 8

El resultado del sorteo dependía solo de los valores revelados. Como se revelan en público y de a uno, el último en revelar podía calcular el resultado antes de mandar su revelación, y no revelar si no le convenía.

QUÉ ESTABA MAL

La entropía (ver cap. 3) era la mezcla de los valores revelados, y nada más. El invariante 8 de la spec dice: "nunca debe pasar que el resultado se pueda calcular antes de que termine la fase de revelación". Pero el último en revelar ya conoce todos los demás valores (están en la cadena) y el suyo: puede calcular el resultado con y sin su revelación.

CÓMO SE EXPLOTABA

Ejemplo: Carla se postula desde dos direcciones (C1 y C2). Espera a que todos revelen, calcula los cuatro escenarios (revelar C1, C2, las dos o ninguna) y manda solo las revelaciones con las que gana. Las direcciones que no revela pierden el depósito, pero si la entrada vale más que el depósito, le conviene.

En la demo contra el código subido, la **predicción hecha solo con lo revelado coincidió exactamente con la entropía final** (`0x91afe948ee002b31…` en las dos).

LA CORRECCIÓN: EL CIERRE EN DOS PASOS

DESPUÉS · EN CERRAR()

```
// Paso 1 (vencida la revelación): fijar el bloque siguiente
if (bloqueSorteo == 0) {
    bloqueSorteo = block.number + 1;
    emit CierreSolicitado(bloqueSorteo);
    return;
}
// Paso 2: ya minado ese bloque, sumar su hash
if (block.number <= bloqueSorteo) return;                  // todavía no existe
bytes32 hashBloque = blockhash(bloqueSorteo);
if (hashBloque == bytes32(0)) { bloqueSorteo = block.number + 1; ... return; }  // >256
bloques
uint256 entropia = uint256(keccak256(abi.encodePacked(_entropia, hashBloque)));
... // Fisher-Yates (F1)
```

La clave es el orden de los hechos:

1. El último postulante tiene que decidir si revela **antes de que venza el plazo de revelación**.
2. El bloque del sorteo se elige recién **después** de ese plazo, en el primer `cerrar()`.
3. Entonces, cuando el último decide, ese hash **ni siquiera está elegido**. No puede calcular el resultado: el invariante 8 se cumple.

POR QUÉ ESTA MITIGACIÓN (LA JUSTIFICACIÓN QUE PIDE EL CURSO)

Opción (de la spec del instructor)	Qué hace	Por qué no alcanza o por qué sí
Depósito que se pierde si no revela	Encarece el ataque	Ya estaba. No lo impide: si la entrada vale más que el depósito, conviene igual.
Orden de revelación forzado	Cada uno revela en su turno	Mueve el problema al último del orden.
Muchos participantes	Diluye el sesgo	No lo elimina; depende de algo que el contrato no controla.
Oráculo externo (VRF)	Azar verificable desde afuera	Lo resuelve, pero agrega una dependencia y un servicio pago.
Hash de un bloque fijado después de la revelación (elegida)	Suma un dato que nadie conoce al revelar	**Lo resuelve para cualquier participante, sin dependencias.** Límite: el validador que produce ese bloque podría influir en su hash. Para eso tiene que ser validador *y* participante, una barrera mucho más alta que "ser el último en revelar".

Se mantiene además el depósito que se pierde por no revelar. Con R1, esa plata ahora la puede retirar el organizador en vez de quedar atrapada.

CÓMO LO VERIFICAMOS

- **"Invariante 8: el resultado no se puede calcular antes de que termine la revelación":** era un `skip` del curso y ahora es un test real. Verifica que el bloque del sorteo es posterior a la última revelación, que la entropía final es exactamente `keccak256(lo revelado, hash de ese bloque)` y que **no** coincide con lo que se podía calcular solo con lo revelado.
- **Reprogramación** si nadie completa el cierre en 256 bloques.
- **Demo N2:** antes, la predicción coincidía con la entropía final; después no coincide.

> **Cómo explicarlo:** "Las revelaciones son públicas, así que el último en revelar podía calcular el resultado y decidir si le convenía revelar. Ahora el resultado también depende del hash de un bloque que se elige recién cuando terminó la revelación. Cuando el último decide, ese dato ni siquiera existe."

11.3 N3 · El portero podía marcar entradas el día antes

MEDIA | Contrato: `Entrada.sol` | Funciones: `usar()`, `definirVentanaIngreso()` (nueva)

Tipo: mitigación parcial (oráculo)

> **El portero podía marcar una entrada como usada en cualquier momento. Un portero deshonesto, o alguien con su llave, podía marcar entradas desde su casa el día antes y dejar afuera a sus dueños.**

CONTEXTO

El portero es el **oráculo** del sistema: la cadena no puede saber si una persona cruzó la puerta, solo sabe que el portero lo afirmó. La spec de Puntos lo marca como hueco deliberado y aclara que ninguna mitigación lo elimina.

QUÉ ESTABA MAL Y CÓMO SE EXPLOTABA

`usar()` solo controlaba quién llama y el estado de la entrada. En la demo contra el código subido, el portero marcó una entrada hoy para un evento de mañana. Una vez marcada, al día siguiente la dueña no puede entrar: un segundo `usar()` revierte con "Entrada ya usada".

LA CORRECCIÓN: VENTANA DE INGRESO

```
function definirVentanaIngreso(uint256 inicio, uint256 fin) external soloOrganizador {
    if (inicio >= fin) revert("Ventana invalida");
    inicioIngreso = inicio;  finIngreso = fin;
    emit VentanaIngresoDefinida(inicio, fin);
}

function usar(uint256 tokenId) external soloPortero {
    if (finIngreso != 0 && (block.timestamp < inicioIngreso || block.timestamp >
finIngreso))
        revert("Fuera del horario de ingreso");
    ...
}
```

> **Es una mitigación, no una solución:** dentro del horario del evento el portero todavía puede mentir (marcar a alguien que no vino). La spec nombra otras mitigaciones que no se implementaron. El doble portero cambia el flujo de la puerta para todos. El límite de marcaciones por minuto puede frenar una puerta real en hora pico, y elegir ese número es una decisión de operación. La ventana es **opcional**: si el organizador no la define, `usar()` funciona como antes. Ver el capítulo 15.

CÓMO LO VERIFICAMOS

- **Test de la ventana:** antes de la apertura revierte; durante el evento marca; terminado el evento revierte otra vez.

- **Test de `definirVentanaIngreso`:** solo el organizador, rechaza inicio ≥ fin y emite el evento.
- **Demo N3:** antes "marcada como usada hoy: Alice no va a poder entrar mañana", después "Fuera del horario de ingreso".

> **Cómo explicarlo:** "El contrato no puede saber si alguien entró de verdad: le cree al portero. Lo que sí podemos hacer es acotarlo: ahora solo puede marcar durante el horario del evento, así ni un portero deshonesto ni alguien que le robe la llave puede 'quemar' entradas el día antes."

11.4 R1 · Plata atrapada en el sorteo

MEDIA | Contrato: `Sorteo.sol` | Función: `retirar()` (nueva) | Tipo: fondos atrapados

El sorteo no le devuelve el depósito al ganador ni a quien no revela, pero tampoco tenía ninguna función para retirarlo: esa plata quedaba en el contrato para siempre.

QUÉ ESTABA MAL Y CÓMO SE EXPLOTABA

No hace falta un atacante: la pérdida está garantizada. En la demo con 3 postulantes y 1 entrada (uno no revela), después de que todos reclamaron quedaban **0,02 ETH sin ninguna forma de sacarlos**: el depósito del ganador y el de quien no reveló.

LA CORRECCIÓN

```
function retirar() external onlyOwner {
    if (!cerrado) revert SorteoNoCerrado();
    // lo que se les debe a los perdedores que todavía no reclamaron, NO se toca
    uint256 monto = address(this).balance - _reembolsosPendientes * deposito;
    if (monto == 0) revert SinFondosParaRetirar();
    (bool ok, ) = owner().call{value: monto}("");
    ...
}
```

Al cerrar, el contrato anota cuántos perdedores revelados tiene que reembolsar (`_reembolsosPendientes`). Cada reclamo de un perdedor descuenta uno. El organizador solo puede retirar **lo que sobra**, así que el invariante 7 (el perdedor recupera su depósito) se sigue cumpliendo aunque el organizador retire primero.

Como ahora el Sorteo guarda plata del organizador, también se le deshabilitó `renounceOwnership` (mismo motivo que F4).

> **Decisión tomada:** esa plata es del organizador. El depósito del ganador funciona como el pago de su entrada, y quien no revela lo pierde como penalidad. Si se prefiriera devolvérselo al ganador, es un cambio chico (ver cap. 15).

CÓMO LO VERIFICAMOS

- **"El organizador retira los depósitos que no se devuelven":** 3 depósitos, retira exactamente 2; el perdedor igual recupera el suyo y el contrato queda en 0.
- **Revierte** antes del cierre, si llama otro y cuando no queda nada.
- **"Quien no revela no puede reclamar y su depósito pasa al organizador":** era otro `skip` del curso ("sin consecuencia definida no hay cierre justo posible") y ahora la consecuencia está definida y probada.
- **Demo R1:** antes "quedan 0,02 ETH (no existe ninguna función para sacarlo)", después "0,0 ETH".

11.5 R3 · Traspaso de dueño a una dirección mal tipeada

MEDIA | Contratos: los cinco | Funciones: `transferOwnership()`, `acceptOwnership()` (nueva)

Pasar el control de un contrato a otra dirección era instantáneo. Con un error de tipeo, el control quedaba en una dirección que nadie maneja, y con él la posibilidad de retirar la plata.

QUÉ ESTABA MAL Y CÓMO SE EXPLOTABA

`Ownable.transferOwnership(nueva)` cambia el dueño en el acto, sin comprobar que del otro lado haya alguien. Una dirección son 40 caracteres hexadecimales: un error de tipeo da otra dirección válida, sin dueño. El efecto es el mismo que F4 (renunciar), pero sin que nadie haya querido renunciar. En la demo, después del traspaso el dueño pasó a ser `0xBa96…3Fd9`, una dirección sin clave conocida.

LA CORRECCIÓN: OWNABLE2STEP

```
import { Ownable2Step } from "@openzeppelin/contracts/access/Ownable2Step.sol";
contract Entrada is ERC721, Ownable2Step { ... }   // lo mismo en Sorteo, Reventa, Canje y Puntos
```

Con `Ownable2Step` (de OpenZeppelin, ya instalado: no es una dependencia nueva) el traspaso tiene dos pasos:

1. `transferOwnership(nueva)` solo **propone**: deja a `nueva` como "dueño pendiente".
2. El traspaso se completa cuando `nueva` llama a `acceptOwnership()`. Si la dirección está mal, nadie la acepta y el dueño actual **sigue siendo el dueño**: puede volver a proponer la correcta.

CÓMO LO VERIFICAMOS

Un test por contrato (5): después de `transferOwnership` el dueño sigue siendo el organizador, `pendingOwner()` es la nueva dirección, y recién después de `acceptOwnership()` cambia. Demo R3: antes "la dirección tipeada", después "sigue siendo el organizador".

> **Cómo explicarlo:** "Pasar el control era como mandar plata a un CBU: si te equivocás en un número, se fue. Ahora es como una invitación: el nuevo dueño tiene que aceptarla, y si la dirección estaba mal, no la acepta nadie y no pasa nada."

11.6 R2 · Un sorteo demasiado grande para cerrarse

BAJA | Contrato: `Sorteo.sol` | Función: `abrirSorteo()`

Después de F1, cerrar cuesta unos 30.000 gas por entrada sorteada. Si el organizador anunciaba, por ejemplo, 100.000 entradas y se anotaban más personas, el cierre nunca iba a entrar en un bloque y los depósitos quedaban atrapados.

```
uint256 public constant MAX_ENTRADAS_POR_SORTEO = 500;
...
if (cantidadEntradas_ > MAX_ENTRADAS_POR_SORTEO) revert DemasiadasEntradas();
```

Con 500 entradas el segundo paso del cierre usa unos 16 millones de gas: la mitad de un bloque de 30M. **Verificación:** test (501 revierte, 500 se acepta) y demo R2 (antes "aceptado" con 100.000; después `DemasiadasEntradas`). Para rifas más grandes, se pueden desplegar varios contratos `Sorteo`.

11.7 R4 · Un operador podía quemar entradas ajenas

BAJA | Contrato: `Entrada.sol` | Función: `quemar()` | Principio: defensa en profundidad

Cualquier operador autorizado podía quemar cualquier entrada de cualquiera, sin pedir permiso. Canje sí pedía el permiso antes de llamar, pero Entrada no lo exigía.

ANTES

```
function quemar(uint256 tokenId) external soloOperadorAutorizado {
    _requireOwned(tokenId);        // solo verificaba que existiera
    ...
}
```

DESPUÉS

```
function quemar(uint256 tokenId) external soloOperadorAutorizado {
    // además de ser operador, hace falta el permiso del dueño sobre esa entrada
    _checkAuthorized(_ownerOf(tokenId), msg.sender, tokenId);
    ...
}
```

Hoy no se podía explotar, porque los operadores son Canje (que valida) y Sorteo (que no quema). Pero si mañana se designa un operador con un bug, o una dirección por error, podría destruir entradas ajenas. Ahora **la regla la garantiza el propio contrato de la entrada**, no la buena conducta de quien lo llama. Como efecto extra, tampoco se pueden quemar entradas que todavía tiene el contrato (eso habría dejado un "fantasma" en la lista de disponibles). **Verificación:** test (sin permiso revierte con `ERC721InsufficientApproval`; con permiso quema) y demo R4.

11.8 R5 · El listado viejo que revivía

BAJA | Contratos: `Reventa.sol`, `Entrada.sol` | Función: `comprar()` de Reventa

Si una entrada listada se iba a otra cuenta y después volvía, el listado viejo revivía al precio viejo, sin que el vendedor lo hubiera vuelto a decidir.

Contexto: listar no pone la entrada en custodia (el test de listar lo deja escrito como decisión: "Listar no escrowa el NFT"). Por eso `comprar` verificaba que el vendedor siguiera siendo el dueño. Pero si la entrada se va y vuelve, el dueño vuelve a ser el mismo.

Cómo se explotaba: el vendedor lista con una autorización general (`setApprovalForAll`), le pasa la entrada a otra cuenta y la recupera. El listado viejo sigue activo y alguien lo compra al precio viejo. En la demo: "Bob compró por el listado viejo, al precio viejo".

DESPUÉS

```
// Entrada cuenta los cambios de dueño entre personas
mapping(uint256 tokenId => uint256) public transferencias;     // se incrementa en _update

// Reventa: el listado guarda el número al listarse...
transferencias: entrada.transferencias(tokenId),
// ...y al comprar exige que no haya cambiado
|| entrada.transferencias(listado.tokenId) != listado.transferencias  =>  ListadoInactivo
```

Se eligió el contador, y no la custodia, para respetar la decisión de diseño de no custodiar.
Verificación: test de Reventa (listado viejo revierte con `ListadoInactivo`), test del contador en Entrada y demo R5.

11.9 N6 · La autorización de puntos que nunca se gastaba

BAJA | Contrato: `Canje.sol` | Función: `canjear()` | Tema: approve infinito

El canje revisaba que el titular hubiera autorizado sus puntos, pero no descontaba esa autorización: un permiso dado una vez valía para todos los canjes futuros.

ANTES

```
puntos.quemar(msg.sender, costo);   // quema directo con el rol de quemador: el allowance no se toca
```

DESPUÉS

```
// transferFrom DESCUENTA la autorización; después se queman desde el canje
if (!puntos.transferFrom(msg.sender, address(this), costo)) revert
AutorizacionInsuficiente();
puntos.quemar(address(this), costo);
```

¿Por qué importa, si hoy no era explotable? `canjear` solo quema los puntos de quien la llama, así que nadie podía gastar los puntos de otro. Pero la autorización era decorativa: verificada y nunca consumida. Es el problema del "approve infinito" de la sesión 4: un permiso que no se agota sigue vivo el día que el contrato tenga un bug. Ahora una autorización por el costo exacto se consume en el canje. Quien igual autorice el máximo posible, lo elige a propósito: así funciona el estándar ERC-20.
Verificación: test ("las autorizaciones previas no deben ser infinitas", antes un `skip`: el allowance queda en 0) y demo N6 (antes "queda: 100", después "queda: 0").

12. Lo que sigue sin poder corregirse

Una revisión honesta también dice qué no se pudo resolver, y qué límites tienen las propias correcciones.

N4 · Pagar por fuera para burlar el techo de reventa DISEÑO

El vendedor publica a precio de lista y el comprador le paga la diferencia en efectivo o por transferencia bancaria. Ningún contrato puede ver pagos fuera de la cadena. Es el remate deliberado de la sesión 4. Restringir las transferencias de la entrada no lo resolvería (el pago por fuera sigue siendo posible) y además contradice la decisión de la spec de que la entrada es un ERC-721 transferible. Lo que sí aporta el contrato es dejar rastro público de cada reventa.

N5 · Front-running entre compradores DISEÑO

Dos personas quieren el mismo listado y una paga más propina para pasar primero (cap. 3, "mempool"). Es un problema de infraestructura (MEV), no del contrato, y no hace perder plata: el precio es exacto y quien pierde la carrera ve su compra revertida. El front-running del organizador sobre la comisión (F3) sí se corrigió, porque ahí había plata en juego.

Límites de las propias correcciones

Corrección	Límite que queda
N1 y N2 (hash de un bloque futuro)	El validador que produce ese bloque puede influir en su hash. Hace falta ser validador, no solo participante.
N1 y N2 (256 bloques)	Si nadie completa la operación en unos 51 minutos, se toma un bloque nuevo. Un atacante que vio un hash desfavorable podría esperar eso, pero no controla que nadie más confirme.
N1 (emisiones)	Un lote emitido mientras hay compras pendientes cambia la lista de disponibles.
N3 (ventana)	Dentro del horario el portero sigue siendo un oráculo: puede mentir. Y la ventana es opcional.
N6 (allowance)	Un approve por el máximo sigue siendo infinito, por decisión del usuario.
R2 (tope)	500 entradas por sorteo; para más, varios contratos.

Otros hallazgos documentados que no son de seguridad

- **"01" y "1" cuentan como asientos distintos**, y también "A" y "a". Decidir qué nombres son el mismo asiento físico depende de cómo nombra los lugares cada recinto. Normalizarlo rompería un caso que la spec acepta (el asiento "0X").
- `emitir` **ignora filas sobrantes:** los helpers de los tests dependen de ese comportamiento.
- **La integración Entrada → Puntos no existe:** `Entrada` está designada como emisora de puntos pero nunca los emite. Es una funcionalidad pendiente (el test está como `skip`), no una falla.

13. Evidencia de ejecución

El profesor pidió evidencia de que los contratos se corrieron. Hay cuatro niveles, y cada uno tiene su salida completa guardada en `docs/evidencia/`, con fecha y comando:

Evidencia	Qué demuestra	Archivo
Despliegue con Ignition en un nodo local	Los 5 contratos y sus 4 designaciones cruzadas se despliegan y funcionan juntos	`03-despliegue.txt` · `despliegue.png`
Recorrido de punta a punta (`verificar.ts`)	`emitir → comprar (reservar y confirmar) → usar` anda contra el nodo; los controles de acceso responden	`04-verificar.txt` · `verificar.png`
Demo de ataques, antes y después	Los 15 ataques funcionan contra el código subido y quedan bloqueados contra el corregido	`06-demo-antes.txt`, `05-demo-despues.txt` · `demo-antes.png`, `demo-despues.png`
Tests, antes y después	Los tests nuevos fallan con los contratos sin corregir y pasan con los corregidos	`01-tests-antes.txt`, `02-tests-despues.txt` · `tests-antes.png`, `tests-despues.png`

Cómo se hizo la demo de ataques

El script `scripts/demo-ataques.ts` reproduce los 15 hallazgos. Despliega contratos nuevos para cada escenario y, para cada uno, imprime si el ataque **funcionó** o quedó **bloqueado**. Ese veredicto sale de lo que pasa en la cadena, no de una etiqueta. Se corrió dos veces contra el mismo nodo:

- **Antes:** en una copia del repositorio en el commit `fb59636`, el código tal como se subió.
- **Después:** con el código corregido.

El mismo script funciona en las dos versiones porque detecta qué funciones existen; por ejemplo, solo confirma la compra si existe `confirmarCompras`.

> **Sobre las capturas:** son imágenes generadas a partir de la salida real de cada comando (los `.txt` de `docs/evidencia/`), con estilo de terminal. Para una captura de tu propia pantalla, levantá el nodo (`npx hardhat node`), desplegá (`npx hardhat ignition deploy ignition/modules/SistemaPuerta7.ts --network localhost`) y corré `npx hardhat run scripts/demo-ataques.ts --network localhost` en `1-hardhat/`.

> **Un número que varía entre corridas:** en F1, la diferencia de gas entre 3 y 18 postulantes dio 0 en una corrida y 7.600 en otra. Fisher-Yates elige una posición al azar, y cuando cae sobre la misma casilla el intercambio cuesta menos. Por eso el costo varía unos miles de gas, pero **no crece con los postulantes**: antes la diferencia era de 104.000. El test exige que sea menor a 15.000.

Demo · ANTES (código subido, `fb59636`): 15 de 15 ataques funcionaron

```
$ DEMO_VERSION="ANTES (fb59636)" npx hardhat run scripts/demo-ataques.ts --network localhost

# Demo de ataques contra el código TAL COMO SE SUBIÓ (commit fb59636, sin correcciones), en el nodo local (Hardhat, chain 31337)
# Comando: DEMO_VERSION="ANTES (fb59636)" npx hardhat run scripts/demo-ataques.ts --network localhost   (en una copia del repo en fb59636)

═══════════════════════════════════════════════════════════════
 DEMO DE ATAQUES — versión: ANTES (código subido, fb59636)
 Red: chain 31337 · bloque inicial 356
 Fecha: 2026-10-22T14:03:28.165Z
═══════════════════════════════════════════════════════════════
[F1] DoS del cierre del sorteo: ¿encarecen el cierre los postulantes extra?
     gas de cerrar() con  3 postulantes y 1 entrada: 101030
     gas de cerrar() con 18 postulantes y 1 entrada: 205030
     diferencia: 104000 gas
     → ✗ VULNERABLE: cada postulante encarece el cierre; inflando la lista se supera el límite del bloque y los depósitos quedan atrapados
[F2] El organizador intenta reabrir un sorteo ya cerrado (para volver a sortear)
     abrirSorteo() aceptado; cerrado() = false
     → ✗ VULNERABLE: el sorteo se reabre y se puede volver a sortear hasta que gane quien uno quiere
[F3] El organizador sube la comisión al 100% justo antes de una compra en reventa
     la vendedora lista a 0.1 ETH con comisión 10% (espera cobrar 0.09 ETH)
     la vendedora no tiene nada para retirar (SinFondosParaRetirar())
     → ✗ VULNERABLE: la vendedora cobró 0: el organizador se quedó con todo el pago
[F4] Se llama a renounceOwnership() en Entrada con plata recaudada
     owner() = 0x0000000000000000000000000000000000000000
     retirar(): revirtió (Solo el organizador)
     saldo del contrato: 0.1 ETH
     → ✗ VULNERABLE: el contrato quedó sin dueño y la recaudación atrapada para siempre
[F5] Un postulante manda el doble del depósito al sorteo
     aceptado: 0.02 ETH, pero reclamar() devuelve 0.01 ETH
     → ✗ VULNERABLE: el excedente queda atrapado en el contrato
[F6] Se emite una entrada con una fila que trae JSON
     emitida; la metadata trae 2 campos "image": {"name":"Entrada 1-A","image":"https://sitio-falso.com/x.png-1","description":"Entrada NFT…
     → ✗ VULNERABLE: quien emite puede meter campos en el JSON (o marcado en el SVG) del NFT
[N1] Un contrato atacante compra y revierte hasta conseguir la entrada #3
     consiguió la entrada #3 en el intento 3 (los fallidos solo le costaron gas)
     → ✗ VULNERABLE: el comprador elige su asiento revirtiendo las compras que no le gustan
[N2] El último en revelar calcula el resultado ANTES de que cierre el sorteo
     predicción (solo con lo revelado): 0x91afe948ee002b31…
     entropía final real:               0x91afe948ee002b31…
     → ✗ VULNERABLE: el resultado se puede calcular con lo revelado: el último decide si revela según le convenga
[R1] Depósitos de ganadores y de quienes no revelan, después del cierre
     saldo que queda en el contrato: 0.02 ETH (no existe ninguna función para sacarlo)
     → ✗ VULNERABLE: esa plata no la puede retirar nadie, nunca
[R2] Se abre un sorteo de 100.000 entradas (imposible de cerrar en un bloque)
     aceptado
     → ✗ VULNERABLE: un sorteo así no se puede cerrar nunca si se anotan más que las entradas
[R3] El organizador transfiere el contrato a una dirección mal tipeada
     owner() después del traspaso: 0xBa9616E7D501C4E91793E60a47Bf85C19eBf3Fd9 (la dirección tipeada)
     → ✗ VULNERABLE: el control pasó a una dirección que nadie maneja
[R4] Un operador autorizado (con un bug o malicioso) quema la entrada de alguien
     quemó la entrada #1 de Alice sin su permiso
     → ✗ VULNERABLE: cualquier operador puede destruir entradas ajenas
[R5] La entrada se va y vuelve a la vendedora: ¿revive el listado viejo?
     Bob compró por el listado viejo, al precio viejo
     → ✗ VULNERABLE: un listado olvidado se ejecuta sin que la vendedora lo haya vuelto a decidir
[N3] El portero marca una entrada como usada el día antes del evento
     no existe forma de acotar el horario del portero
     marcada como usada hoy: Alice no va a poder entrar mañana
     → ✗ VULNERABLE: el portero (o quien robe su llave) puede marcar entradas en cualquier momento
[N6] Después de un canje, ¿sigue vigente la autorización de puntos?
     autorización aprobada: 100 · costo del canje: 100 · autorización que queda: 100
     → ✗ VULNERABLE: el permiso dado para un canje queda vivo para los siguientes
═══════════════════════════════════════════════════════════════
 Resumen: 15 ataques funcionaron · 0 bloqueados (de 15)
═══════════════════════════════════════════════════════════════

Captura de la salida real guardada en docs/evidencia/06-demo-antes.txt
```

Demo · DESPUÉS (código corregido): 0 de 15 ataques funcionaron

```
$ DEMO_VERSION="DESPUÉS (corregido)" npx hardhat run scripts/demo-ataques.ts --network localhost

# Demo de ataques contra el código CORREGIDO, en el nodo local (Hardhat, chain 31337)
# Comando: DEMO_VERSION="DESPUÉS (corregido)" npx hardhat run scripts/demo-ataques.ts --network localhost

═══════════════════════════════════════════════════════════
 DEMO DE ATAQUES — versión: DESPUÉS (corregido)
 Red: chain 31337 · bloque inicial 514
 Fecha: 2026-10-22T14:03:30.955Z
═══════════════════════════════════════════════════════════
[F1] DoS del cierre del sorteo: ¿encarecen el cierre los postulantes extra?
     gas de cerrar() con  3 postulantes y 1 entrada: 111137
     gas de cerrar() con 18 postulantes y 1 entrada: 118737
     diferencia: 7600 gas
     → ✓ BLOQUEADO: el costo de cerrar no depende de cuántos se postulan
[F2] El organizador intenta reabrir un sorteo ya cerrado (para volver a sortear)
     abrirSorteo() revirtió: SorteoYaAbierto()
     → ✓ BLOQUEADO: un contrato hace un solo sorteo
[F3] El organizador sube la comisión al 100% justo antes de una compra en reventa
     la vendedora lista a 0.1 ETH con comisión 10% (espera cobrar 0.09 ETH)
     la vendedora retiró ≈ 0.089969652999787571 ETH (neto de gas)
     → ✓ BLOQUEADO: la vendedora cobró lo pactado al listar
[F4] Se llama a renounceOwnership() en Entrada con plata recaudada
     renounceOwnership() revirtió: Renuncia deshabilitada
     → ✓ BLOQUEADO: no se puede renunciar; el organizador conserva el control
[F5] Un postulante manda el doble del depósito al sorteo
     revirtió: DepositoExcedido()
     → ✓ BLOQUEADO: se exige el depósito exacto
[F6] Se emite una entrada con una fila que trae JSON
     emitir() revirtió: Fila invalida
     → ✓ BLOQUEADO: fila y asiento solo aceptan letras y números
[N1] Un contrato atacante compra y revierte hasta conseguir la entrada #3
     10 intentos, ninguno le dio la entrada #3
     → ✓ BLOQUEADO: pagar solo reserva; el asiento sale de un bloque que todavía no existe
[N2] El último en revelar calcula el resultado ANTES de que cierre el sorteo
     predicción (solo con lo revelado): 0x91afe948ee002b31…
     entropía final real:               0x00f657a158f20009…
     → ✓ BLOQUEADO: el resultado depende del hash de un bloque fijado después de la revelación
[R1] Depósitos de ganadores y de quienes no revelan, después del cierre
     saldo que queda en el contrato: 0.0 ETH
     → ✓ BLOQUEADO: el organizador retira lo que no se devuelve y no queda nada atrapado
[R2] Se abre un sorteo de 100.000 entradas (imposible de cerrar en un bloque)
     revirtió: DemasiadasEntradas()
     → ✓ BLOQUEADO: hay un tope (500) que siempre entra en un cierre
[R3] El organizador transfiere el contrato a una dirección mal tipeada
     owner() después del traspaso: sigue siendo el organizador
     → ✓ BLOQUEADO: el traspaso espera a que el nuevo dueño lo acepte
[R4] Un operador autorizado (con un bug o malicioso) quema la entrada de alguien
     quemar() revirtió: ERC721InsufficientApproval("0x9965507D1a55bcC2695C58ba16FB37d819B0A4dc", 1)
     → ✓ BLOQUEADO: hace falta el permiso del dueño sobre esa entrada
[R5] La entrada se va y vuelve a la vendedora: ¿revive el listado viejo?
     comprar() revirtió: ListadoInactivo()
     → ✓ BLOQUEADO: si la entrada cambió de dueño, el listado viejo deja de valer
[N3] El portero marca una entrada como usada el día antes del evento
     el organizador fijó el ingreso para mañana
     usar() revirtió: Fuera del horario de ingreso
     → ✓ BLOQUEADO: solo puede marcar dentro del horario del evento
[N6] Después de un canje, ¿sigue vigente la autorización de puntos?
     autorización aprobada: 100 · costo del canje: 100 · autorización que queda: 0
     → ✓ BLOQUEADO: el canje consume la autorización
═══════════════════════════════════════════════════════════
 Resumen: 0 ataques funcionaron · 15 bloqueados (de 15)
═══════════════════════════════════════════════════════════

Captura de la salida real guardada en docs/evidencia/05-demo-despues.txt
```

Despliegue en el nodo local (extracto final)

```
$ pnpm desplegar (hardhat ignition deploy ignition/modules/SistemaPuerta7.ts --network localhost)

[ SistemaPuerta7 ] successfully deployed 🚀

Deployed Addresses

SistemaPuerta7#Entrada - 0x5FbDB2315678afecb367f032d93F642f64180aa3
SistemaPuerta7#Puntos - 0xe7f1725E7734CE288F8367e1Bb143E90bb3F0512
SistemaPuerta7#Canje - 0x9fE46736679d2D9a65F0992F2272dE9f3c7fa6e0
SistemaPuerta7#Reventa - 0xDc64a140Aa3E981100a9becA4E685f962f0cF6C9
SistemaPuerta7#Sorteo - 0x5FC8d32690cc91D4c39d9d3abcBD16989F875707
```

Captura de la salida real guardada en docs/evidencia/03-despliegue.txt · extracto: líneas finales del despliegue

Recorrido de punta a punta contra el nodo

```
$ pnpm verificar (hardhat run scripts/verificar.ts --network localhost)

# Verificación automática contra el nodo local
# Fecha: 2026-10-22 10:56:35 -0300
# Comando: corepack pnpm verificar   (= hardhat run scripts/verificar.ts --network localhost)

Direcciones que ve el front:
  Entrada   0x5FbDB2315678afecb367f032d93F642f64180aa3
  Puntos    0xe7f1725E7734CE288F8367e1Bb143E90bb3F0512
  Canje     0x9fE46736679d2D9a65F0992F2272dE9f3c7fa6e0
  Reventa   0xDc64a140Aa3E981100a9becA4E685f962f0cF6C9
  Sorteo    0x5FC8d32690cc91D4c39d9d3abcBD16989F875707

Lecturas:
  precio()   = 0.05 ETH
  owner()    = 0xf39Fd6e51aad88F6F4ce6aB8827279cfffFb92266
  portero()  = 0x70997970C51812dc3A010C7d01b50e0d17dc79C8
  puntos     = 0.0
  emisor     = 0x5FbDB2315678afecb367f032d93F642f64180aa3
  quemador   = 0x9fE46736679d2D9a65F0992F2272dE9f3c7fa6e0
  reventa    = entrada 0x5FbDB2315678afecb367f032d93F642f64180aa3
  sorteo     = deposito 0.01 ETH

Emitiendo 3 entradas desde el organizador (filas V0, V1, V2)…
  emitidas()    = 3
  disponibles() = 3
  comprador tiene 1 entrada(s), tokenId 2
  datosAsiento(2) = sector 1, fila V1, asiento 2
  estado(2) = 2 (2 = Vendida)

Marcando la entrada como usada desde la billetera del portero…
  estado(2) = 3 (3 = Usada)

Control de acceso: el portero NO pudo retirar fondos. Correcto.

Aprobando y listando la entrada usada (debe fallar: no se revende usada)…
  EntradaUsada: no se listó una entrada ya usada. Correcto.

✓ Verificación completa.
```

Captura de la salida real guardada en docs/evidencia/04-verificar.txt

14. Cómo se verificó todo (tests)

El método: rojo → corrección → verde

1. Para cada falla se escribió un test que describe el comportamiento **correcto**.
2. Se lo corrió con el contrato viejo: **falló**. Se revisó el motivo; por ejemplo, el atacante de N1 sí consiguió el asiento.
3. Se corrigió el contrato.
4. Se corrió la suite completa de los 5 contratos: todo en verde.

En la segunda ronda, la corrida del "antes" tiene muchas fallas que no son la vulnerabilidad en sí: muchos tests llaman a funciones que todavía no existían, como `confirmarCompras`, o esperan el cierre en dos pasos. Eso es normal en TDD. La demo de ataques es la que muestra cada vulnerabilidad de forma aislada.

La suite completa

	Pasan	Fallan	Pendientes (`skip`)
Al recibir el código (pull)	123	3	12
Después de la primera ronda	135	0	12
Segunda ronda, antes de corregir (rojo)	82	85	7
Final	**160**	**0**	**7**

Hay **34 tests nuevos**: 9 de la primera ronda y 25 de la segunda. Cinco de los `skip` del curso pasaron a ser tests reales porque ahora tienen solución o una consecuencia definida (invariante 8 del sorteo, quien no revela, autorizaciones no infinitas, atomicidad del canje y listar sin aprobación infinita). Los 7 que quedan pendientes son los que no tienen arreglo en el código (oráculo, pagar por fuera, front-running) o los que no son de seguridad (aforo por sector, integración con Puntos, designaciones del despliegue y una prueba de reentrada que el diseño ya evita).

Tests nuevos de la segunda ronda

Contrato	Test	Hallazgo
Entrada	Pagar solo reserva; el asiento se asigna con el bloque siguiente	N1
Entrada	Un contrato atacante no puede comprar y revertir hasta conseguir su asiento	N1
Entrada	El asiento no depende de quién ni cuándo confirma	N1
Entrada	Las reservas se procesan en orden de llegada	N1
Entrada	Sin confirmación en 256 bloques, la reserva se reprograma	N1
Entrada	El portero solo marca dentro de la ventana; `definirVentanaIngreso`	N3
Entrada	Traspaso en dos pasos	R3
Entrada	Un operador no quema sin permiso del dueño	R4
Entrada	Cuenta los cambios de dueño entre personas	R5
Sorteo	Invariante 8; reprogramación del cierre	N2
Sorteo	El organizador retira lo que no se devuelve; casos de revert; quien no revela	R1
Sorteo	Tope de 500 entradas	R2
Sorteo	`renounceOwnership` revierte; traspaso en dos pasos	R1, R3
Reventa	Listado viejo no se compra; listar sin aprobación infinita; traspaso en dos pasos	R5, R3
Canje	Autorización consumida; atomicidad (invariante 7); traspaso en dos pasos	N6, R3
Puntos	Traspaso en dos pasos	R3

Archivos modificados

Archivo	Cambio
`contracts/Entrada.sol`	F4, F6, N1, N3, R3, R4, R5
`contracts/Sorteo.sol`	F1, F2, F5, N2, R1, R2, R3
`contracts/Reventa.sol`	F3, F4, R3, R5
`contracts/Canje.sol`, `contracts/Puntos.sol`	N6 (Canje), R3
`contracts/test/AtacanteCompra.sol` (nuevo)	Atacante "simular y revertir", solo para tests y demo
`test/*.ts`	34 tests nuevos y helpers de compra y cierre en dos pasos
`scripts/verificar.ts`, `scripts/demo-ataques.ts` (nuevo)	Compra en dos pasos y demo de ataques
`2-front/`	Compra y cierre en dos pasos, valor y salt al azar y guardados, retiro del sorteo, textos de errores nuevos, ABIs regeneradas
`docs/spec/*.md`, `docs/COMO-PROBAR.md`	Sección "Cambios tras la revisión de seguridad" y cómo correr la demo

15. Decisiones que tomé (y se pueden revisar)

Algunas correcciones obligaron a elegir entre opciones razonables. Estas son las elecciones y por qué, para que se puedan discutir:

Decisión	Por qué	Alternativa
Los depósitos de ganadores y de quienes no revelan son del organizador (R1)	El depósito del ganador funciona como el pago de su entrada; el de quien no revela es la penalidad que encarece el sesgo del último revelador.	Devolverle el depósito al ganador.
Tope de 500 entradas por sorteo (R2)	Con ~30.000 gas por entrada, el cierre usa la mitad de un bloque de 30M: margen amplio.	Otro número, o cierre por tandas.
La ventana de ingreso es opcional (N3)	Mantiene el comportamiento anterior si no se configura, y no obliga a saber la fecha del evento al desplegar.	Hacerla obligatoria (más seguro por defecto).
Compra: reservas en orden de llegada, confirmables por cualquiera (N1)	Impide que cada comprador elija cuándo asignar la suya; nadie depende de nadie para recibir su entrada.	Oráculo de azar externo (VRF).
Si el bloque todavía no existe, confirmar y cerrar no hacen nada (no revierten)	En el nodo local los bloques solo se minan con transacciones: si revirtieran, la wallet no las mandaría y la cadena no avanzaría nunca.	Revertir con un error explícito.
Contador de transferencias en vez de custodia (R5)	Respeta la decisión de diseño de que listar no custodia la entrada.	Custodiar la entrada en Reventa mientras está listada.
Valor y salt del sorteo se guardan en el navegador (front)	Si se perdían al recargar la página, no se podía revelar y se perdía el depósito.	Descargarlos como archivo.

16. Cómo presentarlo: guía de 6 minutos y preguntas frecuentes

El guion

1. **Contexto (30 s):** "Revisamos los cinco contratos de Puerta 7. En dos rondas corregimos 15 fallas, cada una con un test que primero demostró el problema, y corrimos todo en un nodo local."
2. **Primera ronda, lo grave (1 min 30 s):** el sorteo se podía trabar con mucha gente y congelar los depósitos (F1), y se podía reabrir para volver a sortear (F2). En la reventa, la comisión se podía subir a último momento (F3).
3. **Segunda ronda (2 min):**
   - La compra ahora es en dos pasos, para que nadie pueda comprar, mirar y deshacer (N1).
   - El sorteo suma el hash de un bloque que se elige después de la revelación, así el último en revelar no calcula el resultado (N2).
   - El portero solo marca en el horario del evento (N3).
   - La plata del sorteo ya no queda atrapada (R1) y los traspasos de dueño se aceptan (R3).
4. **Evidencia (1 min):** mostrar las dos capturas de la demo: el mismo script contra el código subido (15 de 15 ataques funcionan) y contra el corregido (0 de 15).
5. **Lo que no se puede (1 min):** pagar por fuera y el front-running no tienen arreglo dentro de un contrato; el portero sigue siendo un oráculo, aunque acotado.

Preguntas que te pueden hacer

¿Por qué la compra ahora tiene dos pasos? ¿No es peor para el usuario?

> Es un clic más, y es el precio de que el azar sea de verdad. Con un solo paso, el número que elige el asiento ya es público y la entrega ocurre en la misma transacción, así que un bot puede deshacer las compras que no le gustan. Con dos pasos, el número sale de un bloque que no existe cuando pagás.

¿Por qué no usaron un oráculo de azar como Chainlink VRF?

> Es una dependencia nueva, cuesta plata y en el nodo local necesita un simulador. El hash de un bloque futuro resuelve el problema contra cualquier comprador o participante. Lo único que no cubre es un validador que además participe, y eso queda documentado.

¿Qué pasa si nadie confirma la compra o el cierre?

> Cualquiera puede hacerlo: los siguientes compradores, el organizador o un bot. Si pasan más de 256 bloques (unos 51 minutos), el contrato toma un bloque nuevo y nada se traba.

¿El organizador no es alguien de confianza? ¿Por qué cuentan F2, F3 y R3?

> El objetivo de un smart contract es no depender de la confianza. La spec del sorteo pide que "nadie" pueda inclinar el resultado, y en la reventa el vendedor acepta una comisión que nadie le debería poder cambiar. Además, la llave del organizador se puede robar o usar mal (R3): un buen contrato limita lo que se puede hacer con ella.

En F3, ¿cómo puede pasar primero el organizador si mandó su transacción después?

> Dentro de un bloque no se ordena por llegada: lo decide el validador que arma el bloque, que cobra las propinas y pone primero a quien paga más. Después de la corrección da lo mismo, porque la comisión queda grabada en el listado.

¿Cómo sé que las correcciones no rompieron otra cosa?

> Pasan los 160 tests de los 5 contratos, incluidos todos los que ya existían. El recorrido de punta a punta (`verificar.ts`) anda contra el nodo, y el front compila contra las ABIs nuevas.

¿Ya está desplegado?

> En el nodo local de Hardhat (chain 31337), donde corrieron el despliegue, la verificación y la demo. No en una red pública.

¿Se corrigió todo?

> Todo lo que tiene arreglo dentro de un contrato. Pagar por fuera de la cadena y el front-running entre compradores no lo tienen, y el portero sigue siendo un oráculo dentro del horario. Está explicado en el capítulo 12.

17. Glosario

Término	Significado
Allowance / approve	Permiso que le das a otra dirección para gastar tus tokens (o mover tu NFT) en tu nombre.
blockhash	El hash (la "huella") de un bloque ya minado. Un contrato solo puede leer el de los últimos 256 bloques.
Commit-reveal	Técnica de dos pasos: primero publicás un "sobre cerrado" (un hash) con tu valor, después lo abrís. Nadie ve tu valor antes de tiempo.
Defensa en profundidad	Que una regla la garantice cada capa, no solo la que llama (R4).
DoS	Denegación de servicio: lograr que una función no se pueda ejecutar.
Entropía	Grado de impredecibilidad. En el sorteo, el número del que sale el resultado. Explicación completa en el cap. 3.
Fisher-Yates	Algoritmo clásico para mezclar una lista al azar, paso a paso, como mezclar cartas.
Front-running	Ver una transacción ajena en el mempool y mandar una propia que se ejecute antes.
Gas	Lo que cuesta ejecutar operaciones. Cada bloque tiene un tope.
Gwei	Una milmillonésima de ether. Es la unidad del precio del gas y de las propinas.
Griefing	Ataque cuyo objetivo es hacer daño, aunque el atacante no gane nada.
Inyección	Meter caracteres especiales en un dato para cambiar el formato del texto donde se pega.
Invariante	Regla que nunca debe romperse.
JSON	Formato de texto para datos estructurados: `{"clave":"valor"}`.
Lista blanca	Validar diciendo qué se permite (en vez de qué se prohíbe).
Mempool	Sala de espera pública de las transacciones que todavía no entraron en un bloque.
MEV	La ganancia que se puede sacar eligiendo el orden de las transacciones dentro de un bloque.
Metadata / tokenURI	La "ficha" de un NFT (nombre, imagen) que leen wallets y marketplaces.
Oráculo	Algo que trae a la cadena un dato del mundo real (como "esta persona entró"). La cadena no lo puede verificar.
Orden de llegada (FIFO)	"El primero que entra es el primero que sale": las reservas se procesan en el orden en que se hicieron.

Ownable / Ownable2Step	Patrones de OpenZeppelin que le dan un dueño al contrato; el segundo exige que el nuevo dueño acepte el traspaso.
prevrandao	Número que trae cada bloque. Parece azar, pero es público y no sirve como secreto.
Priority fee (propina)	Lo que una transacción le ofrece al validador además del costo del gas. A más propina, más adelante suele quedar en el bloque.
Pull (patrón)	No mandarle la plata a nadie automáticamente: cada uno la retira cuando quiere. Evita reentradas.
Reserva	Compra pagada que todavía espera su asiento (N1).
Revert	Cancelar una transacción completa, como si nunca hubiera pasado.
SVG	Formato de imagen escrito como texto (etiquetas como `<text>`).
Sybil	Una persona que actúa con muchas direcciones.
Test de regresión	Test que reproduce un bug para que, si vuelve, se detecte enseguida.
Underflow	Una resta que daría negativa. En Solidity 0.8 cancela la transacción.
Validador	Quien arma un bloque (o le encarga armarlo a un "constructor de bloques"): elige qué transacciones entran, en qué orden, y cobra las propinas.

Informe sobre la rama `gperez` (base `fb59636`) con las correcciones de las dos rondas. Las mediciones de gas se tomaron en el nodo simulado de Hardhat; los umbrales de bloque son estimaciones a partir de esas mediciones. Las salidas completas de cada comando están en `docs/evidencia/`.